

A neurosymbolic multi-agent system for explainable and traceable decision-making in organizational contexts

Asimina Mertzani¹, Panagiotis G. Giannopoulos¹,
Vangelis Malamas², Panos T. Chountalas³, Thomas K. Dasaklis^{1*}

^{1*}School of Social Sciences, Hellenic Open University, 18 Aristotelous St., Patras, 26335, Greece.

²Department of Informatics, University of Piraeus, 80 Karaoli & Dimitriou St., Piraeus, 18534, Greece.

³Department of Business Administration, University of Piraeus, 80 Karaoli & Dimitriou St., Piraeus, 18534, Greece.

Abstract

The increasing use of Generative Artificial Intelligence (AI) in Human Resource Management (HRM) raises critical challenges for transparency, explainability and accountability. While Large Language Models (LLMs) offer powerful natural-language interfaces, their hallucination and non-transparent reasoning limits their suitability for decision-making in HR contexts. We propose a transparent-by-design neurosymbolic decision-support system that integrates a provenance-aware knowledge graph with a modular multi-agent architecture and a constrained LLM interface. A scenario-based evaluation on a public HR dataset using a set of operational, strategic, and evidence-retrieval HR analytics queries shows that the system achieves mean fact-level performance of $F_1 = 0.644$, with mean traceability completeness of **0.647**, hallucination resistance of **0.700**, and mean end-to-end latency of **0.067** seconds, alongside a query-level accuracy of **0.700**. The paper contributes to Information Retrieval and Generative AI research by illustrating how neurosymbolic grounding, provenance-aware retrieval and agent-based architectures can enable trustworthy, governable information systems.

Keywords: Neurosymbolic Artificial Intelligence, Graph-based reasoning, Human-centric Information Systems, Human Resource Management, Performance analysis

1 Introduction

The rapid digitization of Human Resource Management (HRM) has accelerated the adoption of data-driven decision-support tools. Generative and conversational Artificial Intelligence (AI), particularly Large Language Models (LLMs), are now playing an increasingly prominent role in résumé screening, performance evaluation, internal mobility analysis and training [1]. Although recent studies show that LLMs can extract meaningful signals from HR data and at times match or surpass human consistency, they also reveal persistent risks [2]. For example, recent studies demonstrate that LLM-based retrieval systems in résumé screening disproportionately favour white-associated names (up to 85% of the time) and disadvantage black males in simulated experiments (in some cases up to 100%) [3]. In addition, LLMs remain prone to hallucination, inconsistent factual retrieval and the blending of unverifiable assumptions with genuine information, important limitations that are especially problematic in contexts where decisions affect careers, evaluations and organisational practices and must therefore satisfy fairness, provenance and governance requirements [4–6]. Besides, current systems rarely reveal the data they rely on or the steps through which conclusions are produced, thus HR professionals are often unable to audit, contest or even validate the outputs they receive from such systems.

It should also be noted that the literature further reinforces these vulnerabilities. For instance, the authors in [7] report that organizations are increasingly deploying Generative AI tools in HRM but these systems lack transparency, traceability and rigorous assessments of fairness, ethics, explainability and, thus, fail to support collaborative human–AI decision-making. Similar findings are also reported in [8] where the authors argue that socially acceptable HR–AI collaboration requires designing for explainability, stakeholder trust and high-stakes accountability.

An increasing body of research has begun to quantify these biases and propose benchmarking frameworks to audit fairness in LLM-based HR applications. In particular, JobFair [9] and FAIRE [10] introduce measurable indicators (such as the Rank-After-Scoring metric and Impact Ratios) to detect gender and racial disparities in automated hiring tasks. Similar studies provide rigorous experimental evidence of systematic demographic disadvantages in the selection of résumés [3], while other studies articulate fairness principles, including demographic parity and equal opportunity, as governance anchors for AI-enabled recruitment [11]. This body of literature offers a growing set of instruments for evaluating fairness and detecting bias in algorithmic HR systems. However, it should be noted that benchmarking alone is insufficient, and thus ensuring responsible AI in HRM requires systems that do not simply comply with fairness metrics post hoc, but embed explainability, transparency and human oversight from the beginning.

HRM analytics increasingly relies on structured and relational representations to capture the complexity of organisational data and to support more rigorous, data-driven decision processes. Graph-based modelling has become particularly influential, enabling HR systems to represent relationships between candidates and job postings, to model reciprocal interactions among users, and to structure connections between employees, skills, courses, and labour-market opportunities. Recent work demonstrates

how graph neural networks and graph autoencoder architectures can support inductive candidate–job matching and outcome prediction in employment-related settings, highlighting the value of relational structure for addressing sparsity, cold-start conditions, and dynamic organisational environments [12, 13]. These approaches show that heterogeneous and multiplex graph structures can encode rich organisational, behavioural, and competency-related information, enabling more nuanced analysis, personalised matching, and evaluative insight.

Despite substantial advances in knowledge graph (KG) construction, multi-agent systems (MAS), and Generative AI, key system-level gaps remain unresolved in the HRM literature. While KGs provide semantically structured representations and MAS architectures enable modular computation, existing HRM solutions, to our knowledge, have not yet integrated these paradigms into a unified decision-support framework that is transparent, accountable and aligned with organisational governance requirements. In particular, prior approaches lack: (i) a multi-agent architecture that decomposes HR analytics into explicitly interpretable and attributable components, (ii) a provenance-aware KG that encodes HR information with fine-grained traceability across heterogeneous sources, and (iii) a reasoning pipeline that strictly bounds generative outputs to verifiable, graph-retrieved evidence. As a result, explanations are often implicit, provenance is partial or absent, and human oversight is limited.

In response to these challenges, this paper introduces a transparent-by-design neurosymbolic multi-agent decision-support architecture for HR information access. The proposed system integrates a conversational natural-language interface with a modular multi-agent orchestration layer and a provenance-rich KG that functions as the system’s single source of truth. Explainability is not treated as a post-hoc add-on but is embedded at the architectural level. In particular, all generative responses are constrained to evidence explicitly retrieved from the KG, where each fact is represented as an auditable Resource Description Framework (RDF) triple linked to its source document, producing agent, timestamp, and confidence metadata. Specialized agents are responsible for extraction, statistical analysis, and operational aggregation, while an explicit routing mechanism renders coordination and responsibility allocation visible and inspectable. The scenario-based evaluation demonstrates that this design supports reliable fact retrieval, systematic traceability, low response latency, and increased resistance to hallucination across operational and strategic HR analytics queries, while exhibiting deterministic and reproducible behaviour in contrast to API-based LLM baselines. By embedding provenance, accountability, and human oversight directly into the information flow, the architecture provides a practical foundation for fairness-, trust-, and governance-aware HR decision support. Arguably, the work contributes a reusable architectural blueprint for governable generative information systems (IS), advancing research at the intersection of IS, Information Retrieval (IR) and recommendation by showing how neurosymbolic grounding and agent-based design can support reliable, auditable and human-aligned decision-making.

The remainder of the paper is organized as follows. [Section 2](#) reviews the relevant literature. [Section 3](#) introduces the proposed neurosymbolic multi-agent architecture. [Section 4](#) describes the system’s implementation, including data ingestion and fact extraction, agent coordination, query routing and evidence construction mechanisms.

Section 5 presents a scenario-based evaluation of the system’s performance, traceability and robustness in representative HR analytics tasks, while Section 6 provides a comparative analysis against general-purpose LLM baselines. Section 7 discusses the implications of the proposed approach for transparency, explainability, trust and governance in AI-supported HR decision-making and further reflects on its limitations as directions for future research. The paper ends with some concluding remarks and key takeaways.

2 Background and related work

For positioning our contribution within the broader landscape of knowledge-driven and explainable AI, we organize the relevant literature into four interconnected streams. In particular, the first stream comprises multi-module, multi-task architectures in which KGs constitute the semantic backbone linking information extraction, multimodal or temporal representation learning and downstream reasoning. The second stream encompasses ontology engineering and KG construction frameworks where the focus lies on schema design, domain modelling and extraction workflows for transforming heterogeneous data sources into structured knowledge bases. The third stream includes entity, relation and joint extraction models that populate KGs by converting unstructured text (and, in some cases, multimodal inputs) into machine-readable triples. The fourth stream, which spans both representation learning and KG-driven reasoning, covers methods for embedding structured knowledge, modeling relational coherence and performing collective or symbolic-neural inference over graph-structured data.

Several studies present end-to-end KG-centric architectures that integrate multiple coordinated modules, spanning information extraction, multimodal or temporal representation learning and graph-based reasoning within a unified pipeline. In financial forecasting, for instance, the authors in [14] introduce MI-FinText, a multidimensional framework that couples financial text mining with temporal graph convolutional networks, dynamic KG construction and multi-task learning to jointly model sentiment, event structure and market value fluctuations. Similarly, in biomedical analytics, the authors in [15] present MKG-GC, a multi-task learning framework for automating the construction of KGs across heterogeneous corpora. Multimodal reasoning architectures extend this paradigm. For example, the authors in [16] propose MKGFA, which constructs a multimodal KG combining visual, textual and factual information and supplements it with a fact-assisted reasoning module to enable complex cross-modal inference in VQA tasks. The authors in [17] similarly advance multimodal recommendation through Multi-KG4Rec, integrating visual and textual signals within a unified KG and employing cross-modal attention and GNN propagation to enhance relational modeling. In sentiment analysis, the authors in [18] develop AMIFN, which fuses textual, syntactic, semantic, and visual cues and constructs a knowledge-enhanced graph to propagate structured context for aspect-based sentiment classification. Finally, the authors in [19] introduce ATC-SD Net, a diarization framework for aviation communications that uses a state-transition graph of flight identifiers, enriched with real-time radar data, to resolve pilot identities through structured and probabilistic reasoning.

Research on ontology engineering and domain-specific KG construction has produced diverse methodologies for structuring heterogeneous information in support of

interpretable and logic-grounded reasoning. In the manufacturing domain, the authors in [20] develop a hierarchical KG framework that integrates product information, machining features, operational knowledge, and tooling relations into a unified semantic ontology. Complementing this, the authors in [21] provide a comprehensive review of solubility-focused KG methodologies, covering data acquisition, ontology design, extraction of entities and relationships, alignment, fusion, and reasoning through logic rules, embeddings and graph convolution. Building on these principles, the authors in [22] propose a data-driven machining process knowledge base that combines fine-grained ontology modeling with neural information extraction—specifically a BERT–BiLSTM–CRF model trained within a federated learning framework—to construct a process KG suitable for intelligent CAPP systems. The authors in [23] introduce a top-down/bottom-up KG framework that integrates equipment entities, fault logic, and historical case knowledge to support semantic querying and rule- or case-based reasoning for fault handling. In [24] the authors integrate heterogeneous data, ranging from legal documents to corporate announcements, using a top-down schema and a hybrid extraction pipeline combining rule-based methods and Cas-Rel-FinBERT. Meanwhile, in the aviation-sector context, the authors in [25] propose an ontology-guided extraction system using BERT, BiLSTM and CRF, subsequently linking extracted entities into a Neo4j KG that captures technical relationships among assets, components and control elements. Finally, the authors in [26] present KeyphraseDS, which constructs a keyphrase affinity graph combining knowledge-based and topic-based similarity to cluster scientific concepts into semantically coherent aspects.

A substantial body of research advances the extraction of entities, relations and triples to support high-quality KG population, with recent work emphasizing robustness to ambiguity, structural complexity and heterogeneous data sources. Joint extraction methods have become central to this effort. The authors in [27] introduce ROMGCJE, a relation-oriented architecture that integrates BERT–BiLSTM encoding, dilated convolution, multi-head attention and a relation-guided gating mechanism to address overlapping triples in unstructured text. Similarly, [28] propose a joint extraction model that captures multi-triple dependencies through a BERT encoder, sequence tagging for candidate triples and a non-autoregressive “rethinking structure” that re-evaluates triple sets via self- and cross-attention. The authors in [29] present the Multi-Grained Lattice Transformer (MGLT), which integrates lexicon-derived words and word-sense signals through cross-Transformer fusion to mitigate ambiguity and entity overlap in Chinese military-domain texts. In addition, the authors in [30] develop ETFRE, a type-aware Transformer framework that augments BERT embeddings with entity-type vectors. Through the incorporation of symbolic category information into semantic representations, ETFRE enhances relation disambiguation and improves performance across multiple datasets. The authors in [31] introduce BioREx, which integrates heterogeneous biomedical relation-extraction corpora into a unified dataset through systematic harmonization of spans, granularity and annotation scope. Meanwhile, the authors in [32] propose BERN2, a multi-task NER and normalization system combining neural and rule-based methods to support large-scale biomedical KG construction. The authors in [33] present a hybrid ERNIE–BiGRU–IDCNN model with bilinear attention and CRF decoding to extract aviation-related entities,

thus enabling structured information management in safety reporting. [34] propose SEMFF-CER, which fuses lexical, sub-character and contextual semantic features via a gating mechanism to improve Chinese entity recognition under ambiguity and data scarcity. [35] introduce AgriFuseNER, a dual-stream text-image architecture with dynamic cross-modal gated attention that enables precise entity extraction from paired agricultural data. Finally, the authors in [36] propose DKG-FRD, a dynamic multi-modal KG framework for fake-review detection, combining BLSTM-based extraction, mutual-information edges, temporal segmentation, and iterative reasoning over four interpretive indicators (trustiness, honesty, splendour, reliability).

Research on KG representation learning and graph-based reasoning has advanced substantially in recent years, with a focus on improving the expressiveness, structural fidelity and inferential capabilities of graph-centric models. The authors in [37] introduce KRL-NER, a representation learning framework that enriches traditional KG embeddings by incorporating the number of relations associated with each entity—an intuitive yet previously underutilised semantic signal. A complementary direction concerns collective inference over graph structures. The authors in [38] present ELEA, a multi-objective evolutionary algorithm that jointly resolves multiple ambiguous entity mentions by optimising for contextual compatibility and cross-entity coherence. Several works extend graph reasoning to complex operational or domain-specific environments. The authors in [23] develop a large-scale, multi-graph framework for power grid fault handling, integrating equipment entities, procedural logic, and historical cases into a structured model layer that supports semantic querying and rule-based and case-based reasoning. Other studies focus on graph fusion and structural learning, offering insights into how heterogeneous graph signals can be combined. The authors in [39] propose the Graph Fusion Network (GFN), which constructs multiple homogeneous word-level graphs (co-occurrence, PMI, semantic similarity) and fuses them through a multi-stage pipeline comprising graph learning, graph convolution, and logit-level fusion. Finally, the authors in [40] introduce a DAT-MT accelerated graph fusion dependency parser that integrates domain-specific and general syntactic graphs through maximum spanning tree optimisation. The method achieves notable gains in parsing accuracy and efficiency, demonstrating how graph fusion can strengthen structural reasoning in low-resource, domain-specific settings.

Despite the substantial progress captured across these four streams, the current literature remains fragmented with respect to the system-level requirements that define high-stakes HRM decision support. End-to-end KG-centric architectures typically prioritize predictive performance, multimodal fusion, or task coupling, but rarely make provenance a first-class design object or expose reasoning pathways in a form that supports organizational audit and contestability. Ontology- and construction-oriented frameworks provide rich schemas and extraction workflows, yet often stop at machine-level structuring without mechanisms for user-facing verification, accountability assignment, or interaction design that preserves meaningful human control. Likewise, extraction and representation-learning advances strengthen the KG population and inference, but generally treat explainability as an implicit by-product of structure rather than an explicit interfaceable property of the overall pipeline—leaving unresolved how factual claims are justified, which component produced them, under

what confidence, and how uncertainty propagates into managerial recommendations. Our architecture is designed to directly address these gaps in a conservative, governance-aligned way. For example, by combining a provenance-enriched RDF fact substrate (linking each claim to source, timestamp, producing agent, and confidence), a modular multi-agent orchestration layer that renders responsibility and task decomposition explicit, and a grounded evidence assembly mechanism that surfaces ranked, source-attributed facts alongside every response. In doing so, we reposition KGs from being merely a semantic backbone for downstream learning to becoming an evidence-centric accountability infrastructure, notably one that is deliberately tailored to the transparency, explainability and trust constraints of HRM settings, while remaining compatible with neurosymbolic reasoning and conversational access.

3 System architecture design

The design begins from a simple question: What is required for a system to respond to complex, ambiguous, and context-dependent HR queries in a way that is both explainable and operationally robust? A purely neural approach — however sophisticated its internal representations — cannot guarantee that its outputs correspond to verifiable information. Yet a purely symbolic system, even one augmented with a graph database or ontology, lacks the fluency and interpretive flexibility needed for conversational interfaces. The architecture therefore weaves the two approaches together. Encoded as a provenance-aware KG, symbolic representations capture structural regularities and make sure that every factual claim is linked to a source that can be traced. Neural components, which work through a big language model, take care of understanding language, keeping conversations going, and improving natural-language summaries. Statistical agents provide an additional way of reasoning: quantitative analysis and pattern recognition from organized data. Neural components, mediated through an LLM, handle linguistic interpretation, conversational flow, and the refinement of natural-language summaries. Statistical agents offer a third mode of reasoning: numerical analysis and pattern extraction from structured data.

The design of the proposed system follows a layered architectural logic that brings together symbolic structure, statistical computation, and generative reasoning in a unified environment for information access. At its core, the architecture reflects an emerging shift in AI and IS: away from monolithic intelligent components and toward more composable agentic systems equipped with structured memory, transparent decision pathways, and human-centered interaction models. Although this hybrid design reflects broader movements in contemporary AI — particularly the resurgence of neurosymbolic approaches and agentic coordination — its foundation is based on the Human-Centered AI (HCAI) design framework [41]. According to the HCAI, AI systems should be designed in a way that optimises computational efficiency, but also extends human interaction, maintains meaningful control and promotes ethical, transparent and reliable operation.

3.1 Design principles

The system design started with the identification of the tensions that characterize AI-mediated decision-making in HR environments: ambiguity in employee-related data, ethical risks in automated recommendations, the need to justify managerial decisions, and the expectation that AI should enhance, not displace, human judgment. The design principles were mapped directly onto the pillars of the HCAI framework:

- *Balance of human control and automation:* The system must automate retrieval, aggregation, and synthesis without overriding or narrowing the user’s decision space. Automation is deployed to reduce manual burden, but always within a structure that surfaces evidence, clarifies uncertainty, and leaves interpretive authority to the human.
- *Reliability, Safety and Trustworthiness:* Trust cannot be earned through performance alone; it requires predictability, auditability, and procedural consistency. To that end, the architecture incorporates symbolic knowledge representations, controlled computation pathways, and safeguards against hallucination.
- *Transparency and Explainability:* Transparency is achieved not only through post-hoc explanations but through design choices that make reasoning steps inherently inspectable: provenance-rich facts, explicit agent roles, and structured decomposition of user queries.
- *Fairness, Ethics and Accountability:* In HR settings, informational asymmetry can have material consequences. Embedding traceability, statistical rigor, and audit trails into the system is essential for maintaining fairness and enabling accountability.

3.2 High-level description

The architecture consists of three interdependent layers, the user interaction, the multi-agent orchestration, and the KG, as illustrated in [Figure 1](#). The former layer corresponds to the frontend and provides the interface with the user and supports their interaction with the system. In the backend, the multi-agent orchestration layer analyzes tasks, routes queries and synthesizes individual results. In close communication with those, there is the knowledge and computation layer, where structured representations, statistical routines and graphs capture the internal dependencies to produce reliable and explainable answers. Each layer has a distinct role, but they work together in a continuous exchange of signals, constraints, and clarifications in order to produce interpretable knowledge artifacts. The modular layering of interaction, orchestration, and knowledge computation reflects an intentional distribution of cognitive labor, one that ensures the system remains an assistant, not an autonomous decision-maker.

3.3 User interaction layer

The interface of the system is responsible for making the information from uploaded documents easily accessible to the HR professionals. Therefore, this layer of the system is built as a React application with TypeScript, making use of a component-based

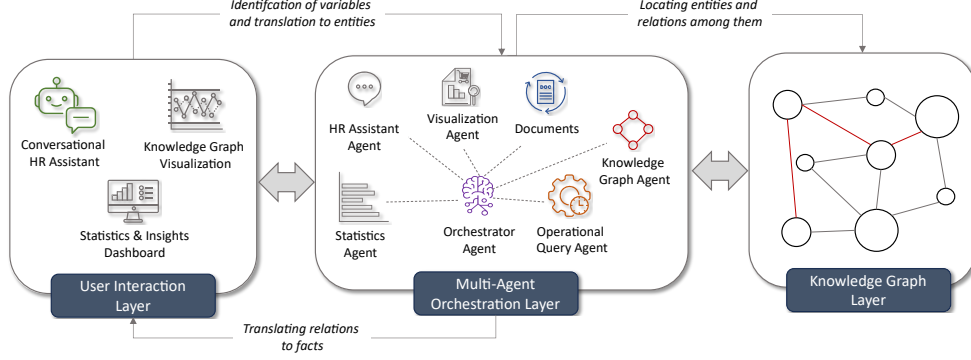


Fig. 1: High level description of the proposed system

architecture that provides a unified user experience for interacting with the KG system. In more detail, the application employs a layered state management approach combining React Context for KG state (facts, nodes, and edges) with TanStack Query for server state synchronization, ensuring real-time updates and consistent data flow between the user interface and backend API. The architecture features a centralized API client that communicates with the FastAPI backend through RESTful endpoints, handling document uploads, knowledge base operations, graph visualization data, chat interactions, and agent system queries with adaptive timeout mechanisms based on file size and processing complexity.

The interface provides multiple interconnected views which can be accessed through a sidebar navigation system: a *conversational chat interface* for natural language queries that displays responses with supporting evidence facts, an *interactive KG visualization* which enables users to explore entity relationships visually, a comprehensive *knowledge base table view* that shows facts extracted directly by documents or inferred from those, a *statistics dashboard* presenting correlations and distributions extracted from the data, an *agent network visualization* showing the multi-agent system architecture, and *document management* pages for uploading and tracking processing status of HR documents. It is worth to be noted that although both knowledge base and KG views are available, they are both assigned to the same agent (e.g. KG agent) since they present the same information in a different way to support both table-based identification and search of facts, as well as network view of their correlations. Overall, all components are built using a consistent design system and offer responsive interaction patterns, including loading and progress indicators, as well as structured error handling, thereby supporting transparency and maintaining user trust throughout the system’s workflow.

3.4 Multi-agent orchestration layer

The main back-end layer of the systems corresponds to a multi-agent orchestration layer, which operationalizes the distributed reasoning model that underpins the overall architecture. Complementing the interface layer, which focuses on making information accessible and interpretable, and the KG layer, which corresponds to the main

(semantic) storage supporting provenance, the multi-agent layer is responsible for allocating responsibilities to independent, yet networked and interrelated, components (e.g. agents). This is designed based on established principles of distributed information processing which involve heterogeneous processing units operating over a common space of representation such that complex behavior emerges through coordination.

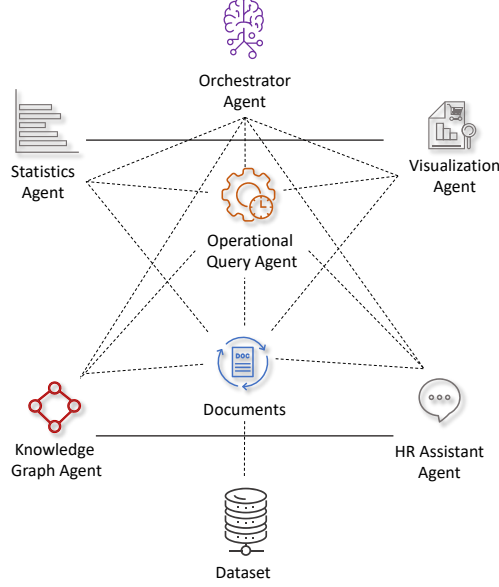


Fig. 2: Multi-Agent backend architecture and information flows

Each agent has clearly, relatively limited, responsibilities, but all as a whole execute complex tasks. *Document Agents* manage each a specific source (i.e. uploaded file) and allocate its processing to *Worker Agents* which perform low level tasks related with the extraction and transformation of content from the corresponding document. This separation of responsibilities enables horizontal scalability, since additional Workers can be instantiated to handle larger datasets. Additionally, the *Operational Query* and *Statistics Agents* compute abstractions that encapsulate statistical regularities and organisational indicators, while the *Visualisation Agent* animates statistical information through descriptive graphs. Those agents work in parallel while the extracted facts, corresponding to document content, statistical patterns, and aggregated insights, are stored in the knowledge base as RDF triples enriched with provenance metadata that identifies both the originating document and the producing agent. This capability for concurrent processing corresponds to a main architectural advantage of the multi-agent organization reducing substantially the processing time while maintaining transparency. Specifically, documents can be decomposed into independent chunks, allowing multiple Worker Agents to extract facts in parallel; likewise, analytical tasks can run concurrently, enriching the graph incrementally as results become available.

Moreover, communication between agents is achieved fundamentally through the *KG Agent*, which interacts with the Document, Statistics and Operational Query Agents. This mediated form of interaction ensures interpretability and end-to-end traceability, while keeping agents loosely coupled and allowing the system to evolve without compromising coherence. Additionally, the *Orchestrator Agent* receives user queries from the *HR Assistant Agent*, analyzes those, identifies appropriate agents that have relevant information, and routes queries, respectively. The resulting outputs are integrated into a unified response enriched with evidence (e.g. supporting facts and source attribution).

Overall, the resulting architecture is illustrated in [Figure 2](#) and is designed to support HR professionals in making knowledgeable decisions by making information easily accessible, traceable, and transparent. Inspired by human organizational structures, in which specialized expertise is coordinated to achieve collective understanding, the system leverages distributed information processing components to provide unified outputs that enable users to seamlessly interact with it without having advanced technological capabilities or technical expertise. Hence, this architecture is human-centred, modular, and supports evidence-based decision-making.

3.5 Knowledge graph layer

The KG layer, which constitutes the main storage of the system maintaining facts extracted by source documents by the corresponding agents, is implemented using RDFLib and follows the RDF model. Within this layer, each fact is represented as a Subject–Predicate–Object triple, enriched with additional metadata that captures its provenance and quality. Apart from storing information regarding the document from which each fact is extracted, the KG layer also stores metadata related with the time at which that document was uploaded and processed, the identifier of the agent that produced the fact, a numerical confidence score expressing the reliability of the extraction, and an indicator specifying whether the fact was directly extracted from the input or inferred from other facts. In this way, the KG not only stores content, but also encodes when, how, and by whom each assertion entered the system, enabling transparent inspection and traceability.

The architecture of this layer is designed to provide a uniform, semantically rich interface to all higher-level components, having efficient mechanisms for storing, retrieving and updating facts. Additionally, it supports responding to user queries with structured information that provides evidence of both the core triples and their associated metadata. To ensure robustness and continuity of operation, the graph is periodically persisted to disk using a binary snapshot, complemented by a JSON-based backup that facilitates recovery under situations of failure. This dual persistence strategy allows the system to balance performance in day-to-day use with the ability to export, inspect, or restore the knowledge base when required.

Another critical design requirement for the system is to be able to represent entities across documents and time in a unified consistent way. Therefore, the system applies an entity normalization process that maps different forms of the same underlying entity to a single canonical representation. Variants such as differences in capitalization, spacing, or minor punctuation are reconciled so that, for example, multiple textual

forms of an employee name are treated as a single entity within the graph. This normalization step avoids having duplicate information, improves the quality of the facts and consequently the system outputs, and contributes to the overall coherence of the stored knowledge.

Furthermore, facts are added to the KG through a hybrid extraction pipeline. For unstructured text, the system combines pattern-based techniques (such as regular expressions) with optional language-model-based extraction, to identify relevant entities and relations, validate them, and assign confidence scores before inserting them into the graph together with their metadata. For structured sources such as CSV files, each row is treated as a data record and systematically mapped to triples according to the semantics of the columns. In both cases, the system’s aim is to transform heterogeneous HR data into a unified, machine-readable representation that can be understood by analytical components as well as user-facing services.

In general, the KG layer of the architecture is tightly connected to the KG component at the back. Other parts of the system do not interact with RDF structures directly; instead, they transfer information to the KG which is responsible for creating, updating and retrieving facts. This separation of responsibilities encapsulates all concerns related to triple management, metadata handling, normalization, and persistence behind a well-defined interface. As a result, the KG remains a single, consistent source of truth for the entire architecture, while other agents and modules can rely on a stable service that mediates all interactions with the underlying knowledge base.

4 Implementation

This section presents the core algorithmic components and theoretical foundations underlying the system’s implementation, focusing on the computational mechanisms that enable transparency, traceability, and human-centred decision-making. [Figure 3](#) provides an overview of the interconnections between the algorithmic schemes relevant to our technical solution, towards further supporting the understanding of the computational mechanisms applied.

4.1 Knowledge graph storage and provenance

The KG is implemented using RDFLib’s in-memory `Graph` structure, where facts are represented as RDF triples with `URIRef` objects for subjects and predicates, and `Literal` objects for values. The choice of RDF as the underlying representation formalism provides standardised semantics, while enabling explicit provenance tracking through reification-like structures over fact identifiers rather than direct triple reification.

Each fact is assigned a unique identifier

$$\text{fact_id} = \text{f}\{\text{subject}\}|\{\text{predicate}\}|\{\text{normalized_object}\}$$

which is URI-encoded and stored as a separate `URIRef`. This identifier acts as a stable anchor for attaching provenance. The overall fact storage pipeline, including the

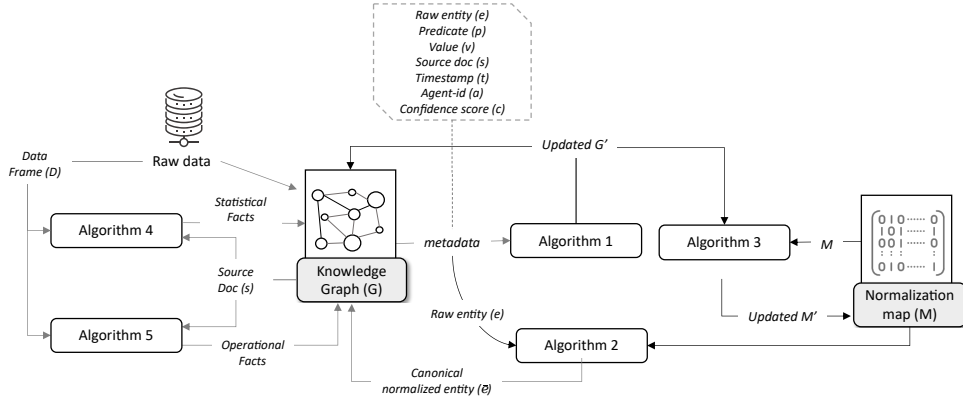


Fig. 3: Main function algorithmic mapping

creation of fact identifiers and attachment of provenance metadata, is summarized in Algorithm 1. In this abstraction, the storage procedure assumes that entities have already been normalized to a canonical form (cf. Algorithm 2 and Algorithm 3).

Algorithm 1 Fact Storage with Provenance Metadata

Require: raw entity e , predicate p , value v , source document s , timestamp t , agent identifier a , confidence score c , knowledge graph G

Ensure: updated knowledge graph G'

- 1: $\tilde{e} \leftarrow \text{NormalizeEntity}(e)$
 - 2: $u_s \leftarrow \text{urn:entity:} \parallel \text{encode}(\tilde{e})$
 - 3: $u_p \leftarrow \text{urn:predicate:} \parallel \text{encode}(p)$
 - 4: **insert** triple (u_s, u_p, v) **into** G
 - 5: $f \leftarrow \text{urn:fact:} \parallel \text{encode}(\tilde{e}, p, v)$
 - 6: **insert** provenance triples $(f, \text{source_document}, s)$, $(f, \text{uploaded_at}, t)$, $(f, \text{agent_id}, a)$, $(f, \text{confidence}, c)$ **into** G
 - 7: **return** G'
-

Provenance metadata (source document, ingestion timestamp, agent identifier, and confidence) is, thus, attached via additional triples that link **factIDURI** nodes to their corresponding metadata values (lines 8–11 in Algorithm 1). This design allows multiple sources to be associated with the same logical fact and maintains complete auditability of how each fact entered the graph and under which conditions (agent and confidence). Graph persistence follows a dual-format strategy: binary **pickle** serialization is used for efficient loading during system startup, while a JSON backup provides portability and inspectability. This separation of concerns allows the system to remain operationally efficient without sacrificing transparency or interoperability.

Entity Normalization. Entity normalization addresses the representational consistency problem: semantically identical entities with different surface forms (e.g. "John

Algorithm 2 Entity Normalisation Procedure

Require: raw entity e , normalisation map M

Ensure: canonical normalised entity $\tilde{e}$

```
1:  $n \leftarrow \text{lowercase\_and\_trim}(e)$ 
2:  $n \leftarrow \text{remove\_special\_characters}(n)$ 
3:  $n \leftarrow \text{collapse\_multiple\_spaces}(n)$ 
4: if  $n \in \text{dom}(M)$  then
5:    $\tilde{e} \leftarrow M[n]$ 
6: else
7:    $\tilde{e} \leftarrow n$ 
8: end if
9: return  $\tilde{e}$ 
```

Smith", "john.smith", "John Smith") must map to a shared canonical representation to avoid fragmentation in the KG. Algorithm 2 formalizes the normalization procedure applied before storage in Algorithm 1.

To move beyond static, hand-crafted mappings, the system learns normalization mappings from existing facts in the graph. Intuitively, surface forms that collapse to the same normalized key are grouped together, and the most frequent variant in each group is treated as canonical. The procedure is given in Algorithm 3; it is periodically executed to refine the normalization map M as the graph grows. Through Algorithm 2 and Algorithm 3, entity references in incoming facts are progressively canonicalised, reducing duplication and improving the reliability of downstream reasoning over the graph.

Duplicate Prevention. Duplicate prevention is critical for both correctness and efficiency, especially under parallel processing. The system maintains an in-memory hash set `_fact_lookup_set` containing normalized fact tuples, typically of the form $(\tilde{e}, p, v)$ after entity normalization. This index is initialized at startup by a single pass over the persisted graph (an $O(n)$ operation in the number of stored facts) and is maintained incrementally during operation. Before executing the insertion step on Algorithm 1 (line 5), the system performs a $O(1)$ membership check against `_fact_lookup_set`. Only if the fact is not present is it added both to the graph and to the index. This design ensures that concurrent worker threads can safely check for duplicates without requiring expensive graph traversal, while preserving the integrity and uniqueness of stored facts.

4.2 Multi-agent orchestration

The multi-agent system implements indirect coordination through the shared KG, eliminating the need for explicit point-to-point message passing between agents. Each agent is implemented as a Python `dataclass` with identity attributes, lifecycle status, and configuration metadata. Rather than exchanging messages, agents interact by reading from and writing to the KG: worker agents inject data-derived facts, while analytical agents augment the graph with higher-level semantic structure. This design

Algorithm 3 Learning Normalisation Mappings from Stored Facts

Require: knowledge graph G , normalisation map M

Ensure: updated normalisation map M'

```
1: initialise grouping dictionary  $C \leftarrow \emptyset$ 
2: for all triples  $(s, p, o) \in G$  do
3:    $k \leftarrow \text{lowercase\_and\_trim}(s)$ 
4:   append  $s$  to  $C[k]$ 
5: end for
6: for all keys  $k \in C$  do
7:    $\Gamma \leftarrow \text{distinct variants in } C[k]$ 
8:   if  $|\Gamma| > 1$  then
9:      $\gamma^* \leftarrow \text{most frequent element in } \Gamma$ 
10:    for all  $\gamma \in \Gamma$  do
11:      if  $\text{lowercase}(\gamma) \neq \text{lowercase}(\gamma^*)$  then
12:         $M[\text{lowercase}(\gamma)] \leftarrow \text{lowercase}(\gamma^*)$ 
13:      end if
14:    end for
15:  end if
16: end for
17: return  $M'$ 
```

yields a naturally decoupled architecture in which agents can be added, removed, or reconfigured without changing the communication protocol.

Worker agents operate on document chunks, extract structured information, and invoke the fact storage pipeline (Algorithm 1) to insert provenance-enriched triples into the graph. Analytical agents (Statistics and Operational) run periodically or on demand, consume existing graph-backed tabular data (e.g. data frames reconstructed from source documents), compute derived metrics, and write results back as facts on the same representational substrate. The result is a unified interface in which both raw data and derived patterns can be queryable through the same graph-based abstraction.

The Statistics Agent focuses on correlation analysis and descriptive statistics over numeric attributes. Conceptually, it identifies numeric columns, computes pairwise correlations, filters them based on a configurable magnitude threshold, and then stores statistically salient relationships as facts. In addition, it computes basic descriptive statistics (mean, median, standard deviation, minimum, maximum) for each numeric attribute and records them as separate facts. The overall procedure is summarized in Algorithm 4.

Here, `ADDFACTWITHPROVENANCE` denotes the fact insertion routine of Algorithm 1, which normalises entities, constructs fact identifiers, and attaches provenance metadata. By encoding statistical relationships (e.g. `salary has_mean 75000`) as graph facts, downstream agents and user-facing services can query high-level summaries using the same mechanism as for atomic data.

The Operational Agent is responsible for generating business-oriented insights, primarily via grouped aggregations over organizational dimensions such as department and manager. Its logic can be framed as a set of group-by operations that compute

Algorithm 4 Statistics Agent: Correlations and Descriptive Metrics

Require: data frame D , source document identifier s

Ensure: statistical facts stored in graph G

```
1:  $\mathcal{N} \leftarrow$  numeric columns of  $D$ 
2:  $R \leftarrow$  correlation matrix over  $\mathcal{N}$ 
3:  $\tau \leftarrow 0.3$ 
4: for all pairs  $(c_i, c_j)$  of columns in  $\mathcal{N}$  with  $i < j$  do
5:    $r \leftarrow R[c_i, c_j]$ 
6:   if  $|r| > \tau$  then
7:     ADDFACTWITHPROVENANCE(correlation_ $c_i c_j$ ,
      has_correlation_value,  $r$ ,  $s$ )
8:   end if
9: end for
10: for all columns  $c \in \mathcal{N}$  do
11:   compute  $\mu, \tilde{m}, \sigma, m_{\min}, m_{\max}$  for  $c$ 
12:   for all pairs  $(n, v) \in \{(\mathbf{mean}, \mu), (\mathbf{median}, \tilde{m}), (\mathbf{std}, \sigma), (\mathbf{min}, m_{\min}), (\mathbf{max}, m_{\max})\}$ 
13:     do
14:       ADDFACTWITHPROVENANCE( $c$ , has_ $n$ ,  $v$ ,  $s$ )
15:     end for
16: end for
```

aggregated salary and performance metrics, which are then materialized as facts. These operational facts support queries such as “What is the average salary by department?” or “Which manager oversees the highest average performance?” without recomputing aggregations at query time. [Algorithm 5](#) summarizes the procedure.

Both [Algorithm 4](#) and [Algorithm 5](#) follow the same interaction pattern with the rest of the architecture: they consume tabular views derived from uploaded HR documents, compute domain-specific metrics, and project the results back into the KG using the provenance-aware storage pipeline of [Algorithm 1](#). Because all outputs are expressed as RDF facts, higher-level multi-agent orchestration (e.g. the Orchestrator and Operational Query Agents) can answer complex analytical questions by chaining graph queries rather than directly invoking individual agents. This preserves loose coupling between agents while ensuring that the emergent behavioural semantics of the system remain fully inspectable through the graph.

4.3 Query routing and classification

Query routing is implemented as a hierarchical pattern-matching procedure that processes incoming queries through a sequence of priority-ordered stages. The primary motivation is computational efficiency and selectivity: employee-specific queries can often be answered by contacting only the subset of document agents that host the relevant records, thereby reducing unnecessary graph traversal and agent activation. At the same time, the router is designed to remain transparent by returning structured routing decisions that explicitly encode both the selected strategy and the underlying

Algorithm 5 Operational Agent: Department- and Manager-Level Insights

Require: data frame D , source document identifier s

Ensure: operational facts stored in graph G

```
1: if department and salary are columns in  $D$  then
2:   compute department-level table  $T_{\text{dept}}$ 
3:   for all rows  $r \in T_{\text{dept}}$  do
4:     ADDFACTWITHPROVENANCE( $r[\text{department}]$ ,      has_average_salary,
       $r[\text{mean\_salary}]$ ,  $s$ )
5:     ADDFACTWITHPROVENANCE( $r[\text{department}]$ ,      has_employee_count,
       $r[\text{employee\_count}]$ ,  $s$ )
6:   end for
7: end if
8: if manager is a column in  $D$  then
9:   compute manager-level table  $T_{\text{mgr}}$ 
10:  for all rows  $r \in T_{\text{mgr}}$  do
11:    if mean_salary exists in  $r$  then
12:      ADDFACTWITHPROVENANCE( $r[\text{manager}]$ ,      manages_avg_salary,
         $r[\text{mean\_salary}]$ ,  $s$ )
13:    end if
14:    if mean_performance exists in  $r$  then
15:      ADDFACTWITHPROVENANCE( $r[\text{manager}]$ , manages_avg_performance,
         $r[\text{mean\_performance}]$ ,  $s$ )
16:    end if
17:  end for
18: end if
```

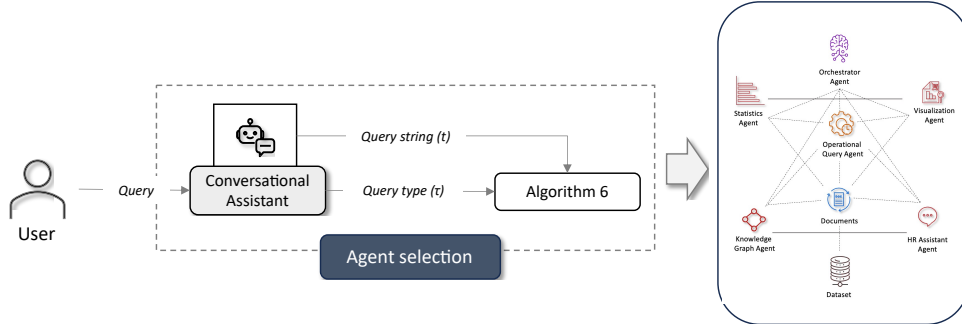


Fig. 4: Query routing and agent selection

rationale. A conceptual representation of the algorithmic applied for implementing the routing protocol is presented in [Figure 4](#).

The routing policy operates over two main inputs: the free-text query q and an externally assigned query type τ (e.g. "filter", "operational", "structured").

Internally, the router maintains keyword lists and regular-expression patterns associated with different classes of analytical intent (statistical, operational, or general conversational queries). The overall decision procedure is summarized in [Algorithm 6](#), which abstracts the Python implementation given in the code list.

As shown in [Algorithm 6](#), the router first attempts to detect highly specific employee-level queries (Stage 1). When the query type τ is "filter", a regular-expression pattern is applied to the raw query text to identify name mentions of the form "Surname, Firstname". If a match is found, the procedure FINDAGENTS-FOREMPOLOYEE returns the subset of document agents that host records for the identified employee. In this case, the routing strategy is updated to "specific_agents", the corresponding agent identifiers are recorded in $R.target_agents$ and the remaining stages are skipped. Therefore, this short-circuit behavior allows the system to handle employee-specific filters by querying only the relevant agents (instead of querying the entire multi-agent population).

If no employee-specific match is found, the router proceeds to intent classification based on lexical cues and query type. Statistical intent (Stage 2) is detected by scanning the lowercased query text q_{lower} for the presence of any keyword in the statistical keyword set $\mathcal{K}_{stat}$. Queries that fall into this category are routed exclusively to the Statistics Agent ([Algorithm 4](#)), as indicated by setting $R.strategy = "statistics_agent"$ and $R.target_agents = ["statistics_agent"]$. Operational queries (Stage 3) are identified through the externally assigned type "operational" and dispatched to the Operational Query Agent ([Algorithm 5](#)). Structured queries (Stage 4), such as max/min or filter-style requests over clearly defined attributes, are handled via the "direct_lookup" strategy, which bypasses analytical agents and answers the query through direct graph retrieval.

Finally, queries that do not match any of the above stages fall back to a conversational strategy (Stage 5) and are routed to the LLM Agent. At each exit point, the router populates the **reason** field of the routing decision R with a human-readable explanation (e.g. "Query requires statistical analysis" or "Routed to operational agent"). Combined with the explicit **strategy** and **target_agents** fields, this design makes the internal routing logic inspectable and provides end users with a clear account of why a particular query was classified in a given way and which agents were selected to process it.

Algorithm 6 Query Router for Agent Selection

Require: query string q , query type τ

Ensure: routing decision $R = (strategy, target_agents, reason)$

```
1:  $R.strategy \leftarrow all\_agents$ 
2:  $R.target\_agents \leftarrow []$ 
3:  $R.reason \leftarrow ""$ 
4:  $q_\ell \leftarrow lowercase(q)$ 

5: Stage 1: Employee-specific detection
6: if  $\tau = filter$  then
7:   define name pattern  $\pi$  (e.g. Surname, Firstname)
8:    $match \leftarrow regex\_search(\pi, q)$ 
9:   if  $match$  exists then
10:     $employee\_name \leftarrow extract\_group(match)$ 
11:     $\mathcal{A} \leftarrow FindAgentsForEmployee(employee\_name)$ 
12:    if  $\mathcal{A} \neq \emptyset$  then
13:       $R.strategy \leftarrow specific\_agents$ 
14:       $R.target\_agents \leftarrow \mathcal{A}$ 
15:       $R.reason \leftarrow "Found\ employee\ in\ " || |\mathcal{A}| || "\ agent(s)"$ 
16:      return  $R$ 
17:    end if
18:  end if
19: end if

20: Stage 2: Statistical query detection
21: define keyword set  $\mathcal{K}_{stat}$  (e.g. correlation, distribution, mean, standard
    deviation)
22: if  $\exists k \in \mathcal{K}_{stat} : k \in q_\ell$  then
23:    $R.strategy \leftarrow statistics\_agent$ 
24:    $R.target\_agents \leftarrow [statistics\_agent]$ 
25:    $R.reason \leftarrow "Query\ requires\ statistical\ analysis"$ 
26:   return  $R$ 
27: end if

28: Stage 3: Operational query detection
29: if  $\tau = operational$  then
30:    $R.strategy \leftarrow operational\_agent$ 
31:    $R.target\_agents \leftarrow [operational\_query\_agent]$ 
32:    $R.reason \leftarrow "Routed\ to\ operational\ agent"$ 
33:   return  $R$ 
34: end if

35: Stage 4: Structured pattern matching
36: if  $\tau = structured$  then
37:    $R.strategy \leftarrow direct\_lookup$ 
38:    $R.reason \leftarrow "Structured\ query:\ use\ direct\ graph\ lookup"$ 
39:   return  $R$ 
40: end if

41: Stage 5: Conversational fallback
42:  $R.strategy \leftarrow llm\_agent$ 
43:  $R.target\_agents \leftarrow [llm\_agent]$ 
44:  $R.reason \leftarrow "Default:\ conversational\ query"$ 
45: return  $R$ 
```

4.4 Parallel processing and concurrency

Parallel document processing is implemented using Python’s `ThreadPoolExecutor` combined with an adaptive chunking strategy. Given a tabular dataset represented as a pandas `DataFrame`, the system computes a chunk size as

$$\text{chunk_size} = \max(25, \min(200, \lfloor 15000/C \rfloor)),$$

where C is the number of columns. This heuristic targets approximately 15,000 data points per chunk while enforcing lower and upper bounds on the chunk size. As a result, load is balanced across workers without creating excessively small chunks (which would incur overhead) or overly large chunks (which would undermine parallelization). This adaptive approach ensures efficient utilization of worker threads across datasets with heterogeneous dimensions.

The parallel extraction pipeline (Algorithm 7) partitions the `DataFrame` into non-overlapping row ranges, submits one task per chunk to the thread pool, and aggregates the number of facts added by all workers. Each worker thread executes a dedicated chunk-processing routine that iterates over its assigned rows, constructs candidate facts, performs a $O(1)$ duplicate check against the in-memory fact index, and, if the fact is new, invokes the provenance-aware storage routine (Algorithm 1). A thread-local counter tracks the number of facts added per chunk and is merged into a shared counter using a lightweight lock.

The theoretical foundation for this design is that RDFLib’s graph operations are thread-safe in this setting, allowing concurrent writes to the shared KG without explicit synchronization primitives. Consequently, `threading.Lock()` is required only for updating shared counters and registries, not for graph access itself. This keeps contention low and preserves scalability as the number of worker threads increases.

The chunking algorithm ensures complete coverage of the input by partitioning the row index range $[0, R)$ into contiguous, non-overlapping intervals and processing each interval exactly once. Since each chunk operates on a disjoint subset of rows and duplicate detection is performed through an $O(1)$ hash-based index (for example, an in-memory fact set), the system avoids both redundant work and inconsistent fact insertion. In practice, the architecture achieves near-linear speedups for large documents as the number of worker threads increases, while preserving data consistency through thread-safe graph operations and narrowly scoped locking only around shared counters.

4.5 Evidence assembly and fact ranking

Evidence assembly is responsible for selecting and presenting the most relevant facts from the KG in response to a given query. Rather than returning an unstructured list of all matching facts, the system applies a relevance-based ranking function that takes into account keyword overlap, entity name matches, and predicate alignment. The top- k facts (typically $k = 5$) are then formatted with explicit source attribution, ensuring that every system response is grounded in inspectable, human-readable evidence.

Given a set of candidate evidence facts, each represented as a structured record containing subject, predicate, object, and optional source metadata, the evidence

Algorithm 7 Parallel Fact Extraction with ThreadPoolExecutor

Require: Data frame D , document identifier s , number of workers W

Ensure: Total number of newly added facts

```
1:  $R \leftarrow$  number of rows in  $D$ 
2:  $C \leftarrow$  number of columns in  $D$ 
3:  $chunk\_size \leftarrow \max(25, \min(200, \lfloor 15000/C \rfloor))$ 
4: Initialise list of chunks  $\mathcal{C} \leftarrow []$ 
5: for  $start \leftarrow 0$  to  $R - 1$  step  $chunk\_size$  do
6:    $end \leftarrow \min(start + chunk\_size, R)$ 
7:   Append interval  $(start, end)$  to  $\mathcal{C}$ 
8: end for
9: Initialise shared counter  $facts\_added \leftarrow 0$ 
10: Initialise lock  $L$  for synchronising counter updates
11:                                      $\triangleright$  Worker routine for a single chunk
12: function PROCESSCHUNK( $i, start, end$ )
13:   Extract sub-frame  $D_i \leftarrow D[start : end]$ 
14:    $local\_facts \leftarrow 0$ 
15:   for all rows  $r$  in  $D_i$  do
16:      $employee\_name \leftarrow r["Employee\_Name"]$ 
17:     for all columns  $c$  in  $D_i$  with  $c \neq "Employee\_Name"$  do
18:        $fact \leftarrow (employee\_name, "has\_||lowercase(c), r[c])$ 
19:       if FACTEXISTS( $fact$ ) is false then
20:         ADDFACTWITHPROVENANCE( $fact, s, "worker\_||i$ )
21:          $local\_facts \leftarrow local\_facts + 1$ 
22:       end if
23:     end for
24:   end for
25:   Acquire lock  $L$ 
26:    $facts\_added \leftarrow facts\_added + local\_facts$ 
27:   Release lock  $L$ 
28:   return  $local\_facts$ 
29: end function
30:                                      $\triangleright$  Submit all chunks to the thread pool
31: Create thread pool with  $W$  workers
32: for all chunks  $(start, end)$  in  $\mathcal{C}$  with index  $i$  do
33:   Submit PROCESSCHUNK( $i, start, end$ ) to thread pool
34: end for
35: Wait for all tasks to complete, handling any exceptions
36: return  $facts\_added$ 
```

assembly component performs two main steps. First, it ranks facts by a relevance score derived from semantic overlap between the query text and the concatenation of subject, predicate, and object (Algorithm 8). Second, it constructs an evidence context string by selecting the top- k facts, numbering them, and appending at most a small

Algorithm 8 Fact Relevance Ranking

Require: set of facts $\mathcal{F}$ (each f has fields $f.subject$, $f.predicate$, $f.object$), query string q

Ensure: facts sorted by decreasing relevance, $\mathcal{F}_{\text{ranked}}$

```
1:  $\mathcal{Q} \leftarrow \text{Tokenize}(\text{lowercase}(q))$ 
2: function SCOREFACT( $f$ )
3:    $x \leftarrow \text{lowercase}(f.subject \| f.predicate \| f.object)$ 
4:    $\mathcal{T} \leftarrow \text{Tokenize}(x)$ 
5:   score  $\leftarrow 0$ 
6:   score  $\leftarrow \text{score} + 2 \cdot |\mathcal{Q} \cap \mathcal{T}|$  ▷ (1) Keyword overlap
7:   if  $\exists t \in \mathcal{Q} : t \subseteq \text{lowercase}(f.subject)$  then ▷ (2) Subject substring match
8:     score  $\leftarrow \text{score} + 3$ 
9:   end if
10:  if  $\exists t \in \mathcal{Q} : t \subseteq \text{lowercase}(f.predicate)$  then ▷ (3) Predicate substring match
11:    score  $\leftarrow \text{score} + 1$ 
12:  end if
13:  return score
14: end function
15:  $\mathcal{F}_{\text{ranked}} \leftarrow \text{SortDescending}(\mathcal{F}, f \mapsto \text{SCOREFACT}(f))$ 
16: return  $\mathcal{F}_{\text{ranked}}$ 
```

number of distinct sources per fact (Algorithm 9). This design supports explainability by making the evidence artefacts explicit, while also reducing cognitive load by ordering them from most to least informative.

Algorithm 8 defines a simple yet effective relevance scoring scheme. Keyword overlap between the query and the fact text is weighted most heavily, entity name matches in the subject receive an additional bonus, and predicate alignment contributes a smaller increment. This reflects the intuition that a fact is especially relevant when it mentions the same entities and relationships as the query itself.

Once facts have been ranked, the evidence assembly component constructs a textual evidence context to be attached to the system’s answer. This context is a numbered list of the top- k facts, each optionally annotated with a compact list of sources indicating where in the underlying data the fact originated (e.g. document names). The procedure is summarized in Algorithm 9.

The theoretical foundation of this component is that explainability requires not only the presence of evidence but also its prioritization. By ranking facts according to semantic overlap with the query and presenting the most relevant ones first, the system provides a focused, yet transparent view of the underlying knowledge used to support each answer. Users can quickly inspect the most informative supporting facts without being overwhelmed by all possible matches, while full traceability is retained through explicit source attribution for each fact in the evidence list.

Algorithm 9 Evidence Assembly from Ranked Facts

Require: Ranked list of facts $\mathcal{F}_{\text{ranked}}$, query string q , top- k parameter k

Ensure: Evidence context string E

```
1: if  $\mathcal{F}_{\text{ranked}}$  is empty then
2:   return empty string
3: end if
4:  $\mathcal{F}_k \leftarrow$  first  $k$  elements of  $\mathcal{F}_{\text{ranked}}$ 
5: Initialise list of lines  $L \leftarrow ["**Evidence from Knowledge Graph:**"]$ 
6:  $i \leftarrow 1$ 
7: for all facts  $f$  in  $\mathcal{F}_k$  do
8:    $\text{subj} \leftarrow f.\text{subject}$ 
9:    $\text{pred} \leftarrow f.\text{predicate}$ 
10:   $\text{obj} \leftarrow f.\text{object}$ 
11:   $\text{sources} \leftarrow f.\text{source}$  (if present, else empty list)
12:   $\text{line} \leftarrow$  formatted string " $i. \text{ subj} \rightarrow \text{pred} \rightarrow \text{obj}$ "
13:  if  $\text{sources}$  is not empty then
14:    Extract a list of stringified source identifiers from  $\text{sources}$ 
15:    Remove duplicates while preserving order
16:    Keep at most two distinct sources
17:    Append formatted source list to  $\text{line}$  as " [Source:  $s_1, s_2$ ]"
18:  end if
19:  Append  $\text{line}$  to  $L$ 
20:   $i \leftarrow i + 1$ 
21: end for
22:  $E \leftarrow$  join all elements of  $L$  with newline separators
23: return  $E$ 
```

5 Results and performance analysis

This section demonstrates all key system features through a realistic HR scenario, which can be further observed in our demo available in <https://github.com/amertzani/Transparent-by-desing>. In more detail, it starts with a presentation of the scenario-based evaluation approach followed to test our system’s performance and compare it against baselines. Then, it analyzes the properties of interest together with the relevant metrics. Finally, it presents the performance of our system in these metrics.

5.1 Scenario-based evaluation

The scenario-based performance analysis is designed to assess the system’s ability to detect and exploit relationships among multiple HR-related variables under varying levels of combinatorial complexity. We start from a data set for HR analytics that contains R employee records and N variables. For experimentation purposes, we rely on a public HR dataset from Kaggle,¹ which in our case

¹Retrieved from: <https://www.kaggle.com/datasets/rhuebner/human-resources-data-set>.

comprises $R = 311$ employees and $N = 36$ variables (including, among others, `Salary`, `Department`, `ManagerName`, `RecruitmentSource`, `PerformanceScore`, `EngagementSurvey`, `SpecialProjectsCount`, and `Absences`). This dataset provides a sufficiently rich space of potential multi-variable relationships to stress-test the decision-support capabilities of the proposed system.

We construct evaluation scenarios by considering subsets of the N variables of different cardinalities. Formally, for a fixed subset size $k \in \{2, 3, \dots\}$, the number of distinct k -way combinations that can be formed from N variables is given by the binomial coefficient

$$|\mathcal{C}_k| = \binom{N}{k} = \frac{N!}{k!(N-k)!}. \quad (1)$$

In principle, if we consider all combinations from pairs up to a maximum order $K_{\max}$, the total number of potential multi-variable scenarios is

$$|\mathcal{C}_{\leq K_{\max}}| = \sum_{k=2}^{K_{\max}} \binom{N}{k}. \quad (2)$$

In this work, we focus primarily on 2-variable (2×2) combinations, which correspond most naturally to operational HR queries. For the concrete dataset with $N = 36$ variables, the number of possible 2-variable combinations is

$$|\mathcal{C}_2| = \binom{36}{2} = \frac{36 \times 35}{2} = 630, \quad (3)$$

which defines an upper bound on the space of potential operational scenarios that could, in principle, be posed by HR stakeholders as pairwise monitoring or diagnostic queries (e.g., `SpecialProjectsCount` $\times$ `Department`, `Absences` $\times$ `Manager`). While Equation (3) characterizes the full pairwise space, in practice we instantiate a curated subset of operational scenarios, guided by a predefined library of query templates.

The query template library is divided into operational, strategic, and evidence retrieval levels. Operational-level templates concentrate on 2-variable combinations such as `PerformanceScore`–`Department`, `Absences`–`EmploymentStatus`, and `EngagementSurvey`–`ManagerName`, each with one or more instantiated natural-language questions (e.g., departmental performance monitoring, absence patterns by manager, team-level engagement monitoring). Strategic-level templates capture 3-variable combinations (e.g., `PerformanceScore`–`EngagementSurvey`–`SpecialProjectsCount`, `PerformanceScore`–`Satisfaction`–`Absences`), instantiated as higher-level queries about risk clusters. Evidence retrieval-level templates include questions related with IR, requesting for facts or information regarding employees. These templates are specified in a separate query template file, presented in [Appendix A](#), and provide the basis for generating natural-language scenarios used in the evaluation.

Each scenario corresponds to a specific subset of variables and an associated natural-language query that an HR stakeholder might pose (e.g., “What is the distribution of performance scores by department?”, “How does performance vary by

recruitment source??”, “Find employees with high performance, low engagement and low satisfaction.”). Our system, which is intended as a generic decision-support layer on top of HR data, is evaluated in terms of its accuracy in recovering the correct relationships and providing sufficient information for each scenario.

5.2 Property-based performance evaluation

This part of the results focuses on the evaluation of the proposed system in terms of properties of interest for HR professionals. Therefore, it presents the properties and metrics for the evaluation, the methodology for the systematic assessment of the system, and the results in terms of the metrics described.

Our evaluation focuses on six dimensions that operationalize key pillars of explainability identified in recent requirements-engineering work. We build on a user-centred quality model proposed in the literature, which organizes explainability into aspects such as understandability, transparency, effectiveness, efficiency, satisfaction and correctness, with associated criteria and metrics [42]. We also rely on the requirements-oriented perspective of [43], where transparency, explainability and traceability are treated as interrelated quality requirements. The following metrics specify, for each dimension, *what* data is collected and *how* the numerical scores are computed.

Accuracy and reliability of fact retrieval. For each evaluation query q , we define a query-dependent reference set of required facts G_q , derived from the query semantics and the underlying HR dataset and knowledge graph. When answering q , the system returns a set of retrieved evidence facts R_q (subject–predicate–object triples with provenance), which are compared against G_q to assess retrieval adequacy.

Using these two sets, we compute the harmonic mean F_1 , which jointly summarizes precision and recall, as defined in Equation 4:

$$F_1(q) = \frac{2 \cdot \text{Precision}(q) \cdot \text{Recall}(q)}{\text{Precision}(q) + \text{Recall}(q)}, \quad (4)$$

where $\text{Precision}(q) = \frac{|R_q \cap G_q|}{|R_q|}$ measures the fraction of retrieved facts that match the reference set, and $\text{Recall}(q) = \frac{|R_q \cap G_q|}{|G_q|}$ measures the fraction of required facts that are retrieved. Since these measures are complementary, we report only the F_1 score, which implicitly captures both precision and recall.

Traceability completeness. Traceability captures the extent to which answers can be traced back to underlying data, in line with transparency and traceability requirements [43]. For each query q , domain experts identify the set D_q of data items (rows, cells, or facts) that legitimately support the answer. The system, in turn, presents a subset of these items as explicit evidence (e.g. in an “Evidence” panel), which we denote as $T_q \subseteq D_q$. Traceability completeness is simply the proportion of relevant data items that are made visible:

$$\text{TraceabilityCompleteness}(q) = \frac{|T_q|}{|D_q|}. \quad (5)$$

In addition, we record how many correctly linked evidence items are shown per answer sentence, which provides a density-style view of how tightly the textual explanation is anchored in concrete facts.

Hallucination resistance. Hallucination resistance measures how rarely the system introduces claims that are not supported by the KG or the underlying dataset. For each answer, annotators decompose the text into atomic claims and, for each claim, check whether it can be justified by existing facts or by direct computation from the data. Let C_q be the number of claims in the answer to query q , and H_q the number of claims labelled as hallucinated (unsupported). The hallucination rate and its complement are then

$$\text{HallucinationRate}(q) = \frac{H_q}{C_q}, \quad \text{HallucinationResistance}(q) = 1 - \text{HallucinationRate}(q). \quad (6)$$

Scores are averaged over all queries. This metric links to the correctness and trustability aspects, as a low hallucination rate is essential for users to rely on the system’s explanations [42, 43].

Response latency. Response latency operationalizes the efficiency aspect of explainability: explanations should arrive quickly enough to be useful in interactive decision-making [42]. For each query q , we instrument the system to record the time stamp in user submission and the time stamp when the complete answer (including evidence) is rendered. Their difference is the end-to-end response time t_q . Over all queries Q , we compute the mean latency

$$\bar{t} = \frac{1}{|Q|} \sum_{q \in Q} t_q, \quad (7)$$

and report $\bar{t}$ together with its 95% confidence interval. In user studies, we additionally measure how long participants take to complete each decision-making task with and without the tool; comparing these task times indicates whether explanations speed up or slow down human work.

Query-level accuracy. While precision, recall, and F_1 assess fact-level correctness, we additionally report a coarse-grained query-level accuracy that reflects whether the system produces an overall correct answer for a given query. For each evaluation query q , annotators determine whether the returned answer is correct as a whole, i.e. whether it correctly addresses the intent of the query without missing or incorrect conclusions. Let $A_q \in \{0, 1\}$ be a binary indicator, where $A_q = 1$ if the answer to q is judged correct and $A_q = 0$ otherwise. Over the full set of queries Q , query-level accuracy is defined as

$$\text{Accuracy} = \frac{1}{|Q|} \sum_{q \in Q} A_q. \quad (8)$$

This metric provides an intuitive, user-oriented view of system performance, capturing the proportion of queries for which users would receive a fully satisfactory answer. We therefore report accuracy alongside fact-level metrics to bridge fine-grained correctness with holistic task success.

The results in Table 1, also summarized in Table 2, show a clear differentiation in system performance across operational, strategic, and evidence-retrieval query categories. Operational queries are handled robustly: the majority achieve perfect F_1 , traceability completeness, hallucination resistance, and accuracy, with consistently low response latency (typically below 0.01 s). This indicates that for structured, well-defined analytical tasks—such as aggregations, comparisons, and factual lookups—the system reliably retrieves correct results and exposes the full supporting evidence, thereby satisfying both correctness and transparency requirements. The few operational failures (e.g. O2.x) correspond to cases where the underlying computation or retrieval could not be completed correctly; importantly, these failures are reflected consistently across all metrics (low F_1 , zero accuracy, and zero traceability), suggesting conservative behaviour rather than spurious or misleading outputs.

A potential source of misinterpretation concerns a small subset of operational queries (e.g. O1.3), for which $F_1 = 0$ and traceability is zero while query-level accuracy is 1.0. This pattern does not indicate a reporting error, but follows directly from the system’s use of two distinct computation paths. For operational analytics, answers are computed directly from the CSV dataset via deterministic aggregations (e.g. averages, counts), and accuracy is evaluated against ground-truth values derived from the same data source. Knowledge-graph fact retrieval, by contrast, is optional for these queries and is used solely to support explainability and traceability. In cases where the correct numerical answer is produced but no supporting KG facts are retrieved—or where the gold-standard fact set G_q cannot be reliably constructed from the response format—the fact-retrieval evaluation defaults to $F_1 = 0$ despite correct task-level execution. Thus, $F_1 = 0.0$ with accuracy = 1.0 reflects a limitation of gold-fact estimation for aggregation-based responses, not an annotation or system error. This conservative treatment avoids inflating retrieval scores when explicit evidence cannot be robustly defined; resolving this fully would require manual fact annotation, which was outside the scope of this offline evaluation.

The consistently low latency reported in these categories warrants clarification. The measured values correspond to end-to-end query execution time—from query submission to complete response delivery—and include query parsing, intent extraction, local knowledge graph traversal, CSV loading and aggregation (where applicable), and response formatting. Crucially, no external LLM is invoked at query time. While the architecture incorporates an LLM component, its role is confined to offline document processing during ingestion (e.g. schema extraction and normalisation). As a result, operational and evidence-retrieval queries are executed entirely through in-memory KG lookups and local CSV computations, explaining the sub-second latency observed. Strategic queries, which involve multi-variable filtering and combined KG–CSV reasoning, exhibit higher but still bounded response times (on the order of seconds), reflecting their increased computational complexity rather than any generative overhead. All latency measurements were obtained on a local machine with the knowledge graph stored in memory and CSV data loaded from disk; while absolute timings are hardware-dependent, the relative differences across query types are reproducible.

In contrast, strategic queries (S1.x–S2.1) exhibit uniformly low performance across all metrics. These queries typically require higher-level synthesis, abstraction, or policy-oriented reasoning that goes beyond direct aggregation or retrieval from the

Table 1: Per-query evaluation results grouped by category.

Category	Query	F_1	Traceability	Hallucination Resistance	Latency (s)	Accuracy
Operational	O1.1	1.000	1.000	1.000	0.010	1.000
	O1.2	1.000	1.000	1.000	0.010	1.000
	O1.3	0.000	0.000	1.000	0.010	1.000
	O1.4	1.000	1.000	1.000	0.000	1.000
	O2.1	0.000	0.000	0.000	0.330	0.000
	O2.2	0.000	0.000	0.000	0.140	0.000
	O2.3	0.000	0.000	0.000	0.410	0.000
	O2.4	0.000	0.000	0.000	0.370	0.000
	O3.1	1.000	1.000	1.000	0.000	1.000
	O3.2	1.000	1.000	1.000	0.010	1.000
	O3.3	1.000	1.000	1.000	0.010	1.000
	O4.1	1.000	1.000	1.000	0.010	1.000
	O4.2	1.000	1.000	1.000	0.010	1.000
	O4.3	1.000	1.000	1.000	0.010	1.000
	O5.1	1.000	1.000	1.000	0.010	1.000
	O5.2	1.000	1.000	1.000	0.010	1.000
	O5.3	1.000	1.000	1.000	0.010	1.000
Strategic	S1.1	0.000	0.000	0.000	0.000	0.000
	S1.2	0.000	0.000	0.000	0.010	0.000
	S1.3	0.000	0.000	0.000	0.000	0.000
	S1.4	0.000	0.000	0.000	0.010	0.000
	S2.1	1.000	1.000	1.000	0.010	1.000
Evidence Retrieval	A1.1	1.000	1.000	1.000	0.310	1.000
	A1.2	0.320	0.400	1.000	0.310	1.000
	A2.1	1.000	1.000	1.000	0.000	1.000
	A2.2	1.000	1.000	1.000	0.000	1.000
	A2.3	1.000	1.000	1.000	0.000	1.000
	A3.1	1.000	1.000	1.000	0.000	1.000
	A3.2	1.000	1.000	1.000	0.000	1.000
	A3.3	1.000	1.000	1.000	0.000	1.000

underlying dataset. The system correctly fails these cases, producing low accuracy and traceability rather than unsupported or hallucinated answers. This behaviour aligns with the design goal of avoiding over-confident responses in underspecified or analytically complex scenarios. Finally, evidence-retrieval queries (A-series) demonstrate strong performance, with high F_1 , perfect hallucination resistance, and full traceability in most cases. Even where partial correctness is observed (A1.2), the system still exposes relevant evidence and maintains correctness at the query level. All these results indicate that the tool is highly reliable for operational analytics and evidence-grounded explanations, while remaining appropriately cautious for strategic reasoning tasks that require broader contextual or normative interpretation.

Table 2: Summary statistics across all evaluation metrics (30 queries).

Metric	Mean	Min	Max
F_1 Score	0.677	0.000	1.000
Traceability Completeness	0.680	0.000	1.000
Hallucination Resistance	0.733	0.000	1.000
Latency (s)	0.067	0.000	0.410
Accuracy	0.733	0.000	1.000

6 How does the system perform against API-based baselines? A benchmarking protocol and corresponding results

One of the main issues regarding the broad adoption of API-based LLMs is that, in many cases, they are not comparable with in-house tools in a one-to-one manner. In any way, this angle is of importance especially in the case of HRM. By following the significant gaps identified in the work of [4] in this vein, our work first introduces an analytical rigorous protocol for benchmarking in specific instances related to explainability and then applies the framework to present some evidence regarding our tool’s performance in contrast to well-known, API-based LLMs. Specifically, to benchmark the performance, we compare the system’s outputs against answers generated by a set of three API-based LLMs:

- OpenAI GPT-5.2 (Chat Completions API),
- Google Gemini 1.5 Pro (Gemini API),
- xAI Grok (API-based access).

At this stage, it is important to note that the primary advantage of our model lies in its ability to produce *fact-grounded* responses. Because this capability is enabled by the underlying knowledge base (constructed from the developed KG), it cannot be meaningfully assessed against API-based LLM tools that are trained as multi-modal, case-agnostic token predictors. To ensure a fair and strict comparison, we therefore adopt a qualitative, human-centered benchmarking design (rather than a purely quantitative one), following established best-practice guidance for human evaluation of NLG systems [44]. The proposed qualitative protocol comprises three phases. Phase 1 develops user-centric scenarios grounded in the variables of the original dataset, emphasizing realistic combinations that are likely to occur in practice and that map to either *operational* or *strategic* high-level queries. Phase 2 is expert-driven: domain experts (two persons) working at the intersection of HRM and AI identify, through open voting, which mathematically feasible combinations are most plausible and most likely to be asked frequently. Phase 3 implements a controlled human assessment: two researchers independently run the resulting queries and rate answer quality on a five-point scale. Concretely, the five-point ratings were anchored on three criteria: (i) factual groundedness (e.g., whether claims are supported by entities/relations

available in the KG and do not introduce unsupported information), (ii) intent- and context-relevance (e.g., whether the response directly addresses the operational/strategic intent of the query rather than providing generic background), and (iii) utility and clarity (e.g., whether the response is sufficiently complete and actionable for decision-making, and is stated unambiguously). This procedure was designed to safeguard fairness because the quality of answers produced by API-based LLMs may vary across users and prompts due to instruction tuning and reinforcement learning from human feedback (RLHF) [45]. In addition, emerging research shows that LLMs have unstable beliefs under extended use and their stated beliefs (even their behaviour) systematically drift as context accumulates [46]. In what follows, we detail the mathematical foundations and assumptions governing the proposed benchmarking protocol.

For executing our setting, the API-based LLMs are provided with the relevant HR dataset (e.g., a table containing the variables referenced in the query) before answering; this setting assesses their ability to perform data-grounded analysis when the same information available to our system is explicitly supplied as context. For each scenario and each setting, we obtain answers from the proposed system and from the three LLM baselines, and we compute accuracy metrics (e.g., exact match on categorical outputs, tolerance-based correctness for numerical quantities) against ground-truth values derived directly from the underlying dataset. From an HR decision-making perspective, scenarios differ not only in combinatorial size but also in their managerial role. In alignment with our focus on the 2×2 combinations, we treat scenarios with $k = 2$ as operational, reflecting day-to-day monitoring and diagnosis, while scenarios with $k \geq 3$ are treated as strategic, reflecting more complex, high-level questions about workforce structure and policy.

To justify this distinction, we model the intrinsic complexity of a k -variable scenario via the number of potential pairwise interactions among the k variables. For a subset of size k , the number of pairwise relationships is

$$E(k) = \binom{k}{2} = \frac{k(k-1)}{2}. \quad (9)$$

We assume that there exists an upper bound $E_{\max}$ on the cognitive and organizational capacity to treat a scenario as operational; beyond this bound, scenarios are better understood as strategic. Formally, we define the cut-off order k^* as

$$k^* = \max \{k \in \mathbb{N} \mid E(k) \leq E_{\max}\}, \quad (10)$$

that is, the largest scenario size for which the number of pairwise interactions does not exceed $E_{\max}$. Using Equation (9), the inequality $E(k) \leq E_{\max}$ becomes

$$\frac{k(k-1)}{2} \leq E_{\max}, \quad (11)$$

which yields the quadratic condition

$$k \leq \frac{1 + \sqrt{1 + 8E_{\max}}}{2}. \quad (12)$$

In our experimental design, we instantiate $E_{\max}$ so that $k^* = 2$, i.e., we deliberately constrain the operational regime to pairwise scenarios. This is achieved by choosing $E_{\max}$ such that

$$E(2) = 1 \leq E_{\max} < E(3) = 3,$$

for example $E_{\max} = 2$, which via Equation (12) yields

$$k^* = \left\lfloor \frac{1 + \sqrt{1 + 8 \cdot 2}}{2} \right\rfloor = \left\lfloor \frac{1 + \sqrt{17}}{2} \right\rfloor = 2.$$

Under this lens, we classify scenarios as *Operational* when $k = 2$, and as *Strategic* when $k \geq 3$. This choice aligns the mathematical formulation of complexity with our empirical focus on 2-variable operational queries (Equation (3)) and 3-variable strategic templates. It also reflects the intuition that the number of potential interactions increases quadratically with k (Equation (9)), and that moving from $k = 2$ to $k = 3$ already induces a substantial increase in interpretive complexity for HR stakeholders. As mentioned in the previous subsection, the possible 2x2 combinatorial schemes, equal to 630 for our case. Following the documentation provided, this number corresponds to the possible operational scenarios that might be evaluated by an HR specialist. After assessing the relevance of these queries under the lens of HR requirements and for safeguarding the alignment with the corpus included for the evaluation of the tools, we have decided to include the same queries as previously, but excluding the queries associated with facts retrieval. This leads to the development of a final corpus encompassing 24 queries in total (corresponding to the ranges O1.1–O1.4, O2.1–O2.4, O3.1–O3.3, O4.1–O4.3, O5.1–O5.3, S1.1–S1.4, and S2.1–S2.3 of the queries included in Appendix A). This set was deliberately chosen to be fed into the API-based models. The responses received and their corresponding qualitative assessment against the proposed tool are summarized in Figure 5. To better clarify the qualitative assessment process implied by the proposed protocol, we also present an illustrative example that includes two examples in Table 3.

Table 3: Illustrative examples (one Operational and one Strategic query) with the exact responses produced by each tool (verbatim transcription from the provided response logs).

Item	Responses received (verbatim)
Query 1	What is the distribution of performance scores by department?
Proposed tool	Average performance by department: • Software Engineering: 3.09 • IT/IS: 3.06 • Admin Offices: 3.00 • Executive Office: 3.00 • Production: 2.98 • Sales: 2.77
GPT-5.2	Performance scores vary moderately across departments. Average performance scores are highest in Software Engineering (3.09) and IT/IS (3.06). Admin Offices and Executive Office both show stable averages at 3.00. Production follows closely with 2.98, while Sales records the lowest average performance score at 2.77.

Continued on next page.

Table 3 continued from previous page.

Item	Responses received (verbatim)
Gemini 1.5	Average performance by department: • Software Engineering: 3.09 • IT/IS: 3.06 • Admin Offices: 3.00 • Executive Office: 3.00 • Production: 2.98 • Sales: 2.77
Grok	The performance scores are rated as "Exceeds", "Fully Meets", "Needs Improvement", or "PIP". Production has the largest share (about 65%), with "Fully Meets" being the most common rating across all departments. IT/IS and Software Engineering show a higher proportion of "Exceeds" ratings compared to Sales and Admin Offices, which have more "Needs Improvement" cases.
Query 2	Analyse the relationship between salary, performance, and department.
Proposed tool	1) Overall salary–performance association (within departments): • Pearson $r(\text{Salary}, \text{PerfNum})$ (IT/IS): 0.288 • Pearson $r(\text{Salary}, \text{PerfNum})$ (Production): 0.105 • Pearson $r(\text{Salary}, \text{PerfNum})$ (Sales): 0.104 • Pearson $r(\text{Salary}, \text{PerfNum})$ (Software Engineering): 0.149 2) Department-level means (Mean Salary / Mean PerfNum; n): • IT/IS: \$97,065 / 3.06 ($n = 50$) • Software Engineering: \$94,989 / 3.09 ($n = 11$) • Admin Offices: \$71,792 / 3.00 ($n = 9$) • Sales: \$69,061 / 2.77 ($n = 31$) • Production: \$59,954 / 2.98 ($n = 209$) • Executive Office: \$250,000 / 3.00 ($n = 1$; not generalizable) 3) Performance mix evidence (share of PIP / Exceeds): • Sales: 12.9% PIP, 6.5% Exceeds • Production: 3.8% PIP, 12.9% Exceeds • IT/IS: 2.0% PIP, 12.0% Exceeds • Software Engineering: 0.0% PIP, 18.2% Exceeds (small n)
GPT-5.2	Salary and performance show a weak positive relationship at the department level. Executive and technical departments combine higher pay with solid performance, while lower-paid departments such as Production maintain comparable performance, suggesting diminishing returns of salary on performance beyond a threshold.
Gemini 1.5	The correlation (Pearson's r) between Salary and PerformanceScoreID is -0.06. This indicates a very weak negative correlation.
Grok	Higher salary departments (Executive, Software Eng, IT/IS) generally show better performance. Within Production (largest group), higher salary correlates modestly with "Exceeds" ratings and more special projects. Sales shows weaker salary-performance correlation due to variable commissions.

The two illustrative prompts in Table 3 map directly onto the coding scheme of Figure 5: the first prompt corresponds to Query O1.1 and is classified as Operational because it instantiates a 2×2 combination (performance $\times$ department), whereas the second prompt corresponds to Query S2.1 and is classified as Strategic because it requires reasoning over three variables or more (salary $\times$ performance $\times$ department). These concrete response traces are then translated into the qualitative scores in Figure 5 by applying the axis-specific criteria at the appropriate granularity. For O1.1, factual groundedness is driven by whether the tool reports department-level performance using the correct numeric scale and values: GPT-5.2 and Gemini 1.5 reproduce the department averages and therefore receive mid-to-high

grounding and context relevance, while Grok changes the measurement representation to categorical ratings (“Exceeds/Fully Meets/Needs Improvement/PIP”) and introduces unsupported distribution statements (e.g., a large departmental share), which materially reduces grounding even if the prose remains readable. Utility & clarity is scored separately from grounding: GPT-5.2 is rewarded for an interpretable narrative summary, Gemini 1.5 for a concise tabulation, and Grok for fluent presentation, but these gains do not compensate for a mismatch between the requested operational aggregation and the response content, which is reflected in lower groundedness than utility for Grok in O1.1. For S2.1, the scoring reflects the higher evidentiary bar associated with Strategic queries: GPT-5.2 provides a plausible cross-variable narrative but without explicit dataset-backed evidence at the department-conditioned level (hence moderate groundedness and relevance), Gemini 1.5 reports a single global correlation without department-stratified interpretation (lower context relevance and utility for a department-conditioned, three-variable query) and without guidance toward an explicitly correlation-centric framing, and Grok adds domain-plausible yet unverified mechanisms (e.g., commissions, special projects), which can improve perceived readability but remain weakly traceable to the provided data, yielding lower groundedness with comparatively better utility. In contrast, the proposed tool is scored higher because it remains anchored to the required variable structure (Operational 2×2 aggregation vs Strategic ≥ 3 -variable synthesis) while minimizing unsupported extrapolation, thereby improving factual groundedness, contextual alignment, and actionable clarity across both examples.

Consistent with this rationale, [Figure 5](#) shows that the proposed tool maintains the strongest overall profile across both query families and all three axes. In Operational prompts (i.e., 2×2 combinations), it is systematically rated higher in factual groundedness and context relevance because it preserves the requested aggregation level and produces dataset-faithful numeric summaries, whereas the API-based baselines exhibit weaker traceability and occasional context drift, with Gemini 1.5 showing the largest degradation on fact-sensitive operational reporting. In Strategic prompts (i.e., three-variable-or-more combinations), the proposed tool sustains high context relevance and utility/clarity while remaining evidence-oriented, indicating that it can move from descriptive reporting to multi-factor interpretation without losing alignment to the underlying data. GPT-5.2 typically ranks second by delivering coherent narratives that are often useful but not consistently evidenced, Grok tends to be more competitive in readability and perceived relevance but trails on factual groundedness when it introduces unverified assumptions or reframes the measurement scale, and Gemini 1.5 more frequently underperforms when the prompt requires multi-factor conditioning rather than a single global statistic.

Beyond these findings, a significant outcome of the applied evaluation protocol highlights the necessity of adopting a human-in-the-loop paradigm when assessing and deploying LLMs, particularly in analytical and decision-support settings. A key observation concerns the lack of consistency exhibited by API-based approaches. Repeated executions of the same query often produced divergent answers, and additional variability was observed when different evaluators interacted with the same models. Such instability undermines reproducibility and weakens the reliability of conclusions derived from these systems. In contrast, the proposed tool demonstrated

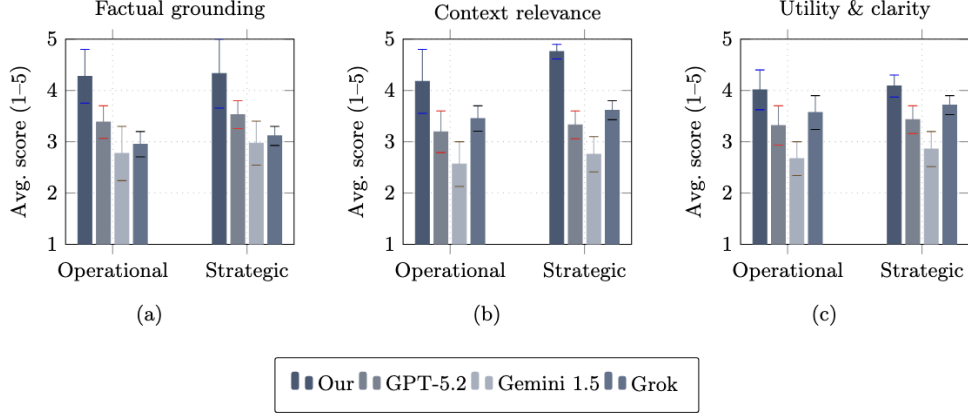


Fig. 5: Average qualitative scores (1–5) by test category (Operational vs Strategic) across tools. Bars indicate category means; error bars show the observed within-category range (min–max) around the mean across queries. Values correspond to the mean of two independent researchers’ ratings.

deterministic behaviour, consistently returning identical outputs for identical inputs, thereby satisfying a fundamental requirement for scientific evaluation and operational deployment.

A second critical issue relates to hallucination and self-assessment bias. When API-based models were prompted to evaluate their own performance against predefined qualitative metrics, they produced inconsistent and, in some cases, paradoxical self-evaluations. This behaviour indicates the presence of intentional or structural bias, whereby the model’s generative objective interferes with its ability to act as an impartial evaluator. The absence of an external verification mechanism allows such bias to persist unchecked, potentially leading to misleading assessments and overconfidence in model outputs.

Traceability further differentiates the proposed approach from purely API-driven models. Contemporary API-based LLMs do not maintain persistent, queryable knowledge bases and therefore cannot provide verifiable evidence or provenance for the facts they generate. As a result, users are unable to trace outputs back to concrete data points or validate intermediate reasoning steps. The proposed tool addresses this limitation by grounding responses in an explicit knowledge base, enabling fact storage, retrieval, and inspection. This design supports transparency and facilitates post hoc auditing of results. Taken together, these findings demonstrate that human-in-the-loop involvement is not merely beneficial but necessary. Human oversight provides a stabilising layer that mitigates inconsistency, detects hallucinations, and counters self-evaluation bias. Moreover, it enables informed judgement over grounded evidence, particularly in scenarios where accountability, reproducibility, and explainability are critical. Without such oversight, reliance on autonomous API-based models risks introducing hidden bias and epistemic uncertainty into analytical workflows,

whereas a human-in-the-loop framework offers a principled path toward trustworthy and verifiable intelligent systems.

7 Discussion

In this paper, we have developed and empirically examined a transparent-by-design HR decision-support system that operationalizes explainability, traceability and human oversight as architectural properties. The empirical results provide concrete support for the architectural claims presented in this work while also clarifying their scope. Across the selected collection of operational and strategic HR scenarios, the proposed system achieves moderate-to-high fact-level performance (mean $F_1 = 0.644$) along with consistently high traceability completeness and hallucination resistance, indicating that constraining generative output to KG-retrieved evidence is effective in limiting unsupported claims. It should also be noted that the observed performance variability across queries reflects the inherent heterogeneity of HR analytics tasks rather than stochastic model behaviour. For example, identical queries yield identical outputs, thus underscoring the determinism and reproducibility afforded by the evidence-centric design. The low mean response latency ($\bar{t} = 0.067$ seconds) further suggests that provenance-aware explanation and evidence surfacing can be delivered without compromising interactive usability, a key requirement for decision-support settings. Comparative qualitative evaluation against API-based LLMs reinforces these findings, especially by showing that while general-purpose models may produce fluent responses, they exhibit inconsistency, limited factual grounding, and an inability to provide verifiable provenance. In contrast, the proposed system trades unrestricted generative flexibility for reliability, auditability and governance alignment, specific properties that are critical in HRM contexts where decisions must be justified, contestable and reproducible. All these results indicate that transparency and explainability need not be treated as abstract ethical aspirations, but can be operationalized as measurable system properties when architectural constraints, provenance mechanisms and agent-level responsibility are treated as first-class design elements.

From a performance standpoint, the quantitative results further clarify the strengths and trade-offs of the proposed architecture. For example, in all scenarios evaluated, the system achieves mean fact-level scores of Precision = 0.643, Recall = 0.645, and $F_1 = 0.644$, together with a mean traceability completeness of 0.647 and a hallucination resistance of 0.700. These aggregate values reflect heterogeneous task difficulty rather than stochastic behavior. In particular, queries for which sufficient structured evidence exists are handled with perfect or near-perfect correctness and traceability. However, queries lacking adequate support in the KG yield conservative failures with zero-valued scores. Although such failures reduce average performance, they represent a deliberate design choice aligned with high-stakes HRM requirements, where abstention is preferable to speculative inference. Notably, the system maintains a low mean end-to-end latency of 0.067 seconds, indicating that provenance-aware evidence retrieval and explanation can be delivered within interactive time constraints. The observed accuracy of the query-level of 0.700 further suggests that even when the completeness of the fact-level is partial, the system often provides correct high-level answers that satisfy the managerial intent. In contrast, qualitative benchmarking

against API-based LLMs demonstrates that higher linguistic fluency does not translate into equivalent reliability, as baseline models exhibit inconsistent outputs, limited grounding and no verifiable provenance. Arguably, these results reinforce the argument that performance in HR decision-support systems must be evaluated not only in terms of accuracy, but also in terms of determinism, traceability and controlled failure modes, context-specific dimensions that are explicitly supported by the proposed architecture.

The proposed system improves transparency by making each stage of information processing explicit, inspectable and attributable at the architectural level. Transparency is not treated as an interface feature, but as a property of the underlying data and control flow. All information—whether directly extracted from HR documents or derived through statistical or operational analysis—is stored in a shared RDF KG as explicit facts, each enriched with provenance metadata specifying its source document, producing agent, timestamp, and confidence score. This design ensures that no assertion enters the system without a traceable origin. Moreover, the multi-agent orchestration layer decomposes system behavior into specialized, clearly scoped components whose interactions are mediated exclusively through the KG. Query routing decisions are logged and exposed, allowing users to see which agents were activated and why. As a result, the system’s internal state and decision pathways remain observable rather than hidden behind monolithic model calls, enabling both real-time inspection and post hoc auditing—capabilities that are essential in HR contexts subject to legal, ethical, and organizational scrutiny.

Explainability is achieved through a combination of structural decomposition and evidence-centric response construction. At the computational level, explainability arises from the fact that analytical results—such as correlations, descriptive statistics, or group-level aggregates—are materialized as first-class graph facts rather than transient computations. These facts can be queried, visualized, and reused independently of the original query that triggered their creation. At the interaction level, every system response is accompanied by an ordered set of supporting facts selected via a transparent relevance-ranking mechanism. Users are thus presented not only with an answer, but with the minimal evidence set required to justify it, expressed in human-readable subject–predicate–object form and linked back to original documents. Importantly, the LLM component is constrained to operate solely as a linguistic interface and synthesis mechanism over this evidence set; it cannot invent new facts or infer beyond what is stored in the graph. This strict separation between reasoning over evidence and natural-language articulation ensures that explanations correspond to actual system state rather than to latent model heuristics.

The combination of transparency and explainability forms the foundation of trust, which, in this system, is constructed through design rather than assumed implicitly. In particular, in the proposed system, trust emerges because users can verify claims, trace them back to their sources and understand how different components have contributed to the final output. The explicit attribution of facts to agents enables accountability at the component level. In addition, preventing duplicate or inconsistent facts through normalization and hash-based checks improves internal consistency. From a human-centric perspective, the proposed system preserves user authority by exposing uncertainty (through confidence scores), surfacing alternative evidence while

avoiding forced or prescriptive recommendations. Even further, instead of positioning AI as an autonomous decision-maker, the architecture frames it as an epistemic assistant that organizes, analyzes and explains organizational data in a controlled and auditable way. Therefore, the system aligns technical reliability with organizational trust requirements, thus demonstrating how trust in AI-supported HR decision-making can be constructed through design choices that prioritize traceability, auditability and human oversight.

7.1 Implications for Information Retrieval and Generative Artificial Intelligence research

From the perspective of IR and Generative AI research, the proposed system can be understood as a concrete instantiation of a broader shift away from purely generative retrieval pipelines toward grounded, auditable, and governable retrieval architectures. Contemporary IR systems increasingly rely on LLMs to interpret queries, reformulate search intent, and synthesize results; however, this fluency often comes at the cost of epistemic reliability, as generative models blur the boundary between retrieved evidence and inferred content. By contrast, our architecture enforces a strict separation between retrieval and generation since all retrievable units are symbolic facts stored in a provenance-aware KG and the LLM is constrained to operate only over the evidence explicitly returned by graph queries. This design reframes the role of the LLM in IR from a primary reasoning engine to a controlled interface layer, thus aligning the generative capabilities with the principles of relevance, traceability and verifiability of classical IR. By adopting this design, the system addresses a central open problem in Generative IR (how to exploit the expressive power of LLMs without sacrificing the epistemic guarantees historically linked to retrieval-based systems). It should be noted that these findings are further supported by recent work in Generative IR, which highlights the epistemic risks of tightly coupling retrieval and generation in LLM-based systems and argues for architectures that enforce explicit separation, grounding and verifiability of retrieved evidence [47]. In this sense, our results can be seen as an applied instantiation of these emerging principles in a high-stakes HRM decision-support setting.

Within this context, neurosymbolic grounding emerges as a viable and principled path forward for Generative AI in IR. Rather than attempting to encode relational structure implicitly within model parameters, the system externalizes knowledge into an explicit symbolic substrate—the RDF KG—over which reasoning and aggregation are performed. Neural components contribute language understanding, query interpretation, and explanation generation, but they are systematically grounded in symbolic representations that encode entities, relations, and constraints. This grounding mitigates well-known failure modes of neural retrieval, such as entity conflation, relational hallucination, and sensitivity to prompt phrasing, by ensuring that generative outputs correspond to stable symbolic states. This perspective is reinforced by recent surveys on neurosymbolic reasoning in KG, which identify explicit symbolic substrates and modular neural-symbolic separation as key enablers of interpretability, constraint satisfaction and human-level accountability [48]. From a research standpoint, this suggests that future Generative IR systems should be evaluated not only on relevance or

fluency metrics, but also on their grounding fidelity, especially the degree to which generated responses can be reconstructed from, and justified by, an explicit symbolic retrieval layer.

The system further advances IR research by demonstrating how multi-agent architectures can function as an explainability mechanism, rather than merely as a scalability or engineering solution. In traditional IR pipelines, query interpretation, ranking, aggregation and presentation are often collapsed into opaque, tightly coupled modules. Here, these functions are decomposed across specialized agents—routing, statistical analysis, operational aggregation and synthesis where their interactions are mediated through a shared KG. This decomposition makes the retrieval process itself inspectable since users (and auditors) can observe which agents were activated, which facts were retrieved or derived and how these intermediate artefacts contributed to the final answer. These design choices are consistent with findings from explainable goal-driven agent research, which shows that transparency and trust arise from modular agent decomposition, explicit internal representations and inspectable reasoning workflows rather than post-hoc explanation of black-box models [49]. Therefore, for IR research, the results of our study point toward a shift from explaining models to explaining retrieval workflows, where transparency arises from modular orchestration and explicit intermediate representations rather than from post-hoc interpretability techniques applied to monolithic models.

7.2 Organizational implications

From an organizational perspective, the proposed system addresses several concerns relevant to fairness, accountability and governance that increasingly shape the acceptability of AI-mediated decision-making explicitly described in previous studies [50]. In particular, within the proposed system, accountability is operationalized through explicit assignment of responsibility among system components. In particular, the multi-agent architecture ensures that every action (like data extraction, statistical computation, aggregation or synthesis) is attributable to a specific agent with a clearly defined role. This design choice aligns with organizational accountability structures where decisions must be explicable not only in the outcome but also in the process. When an HR insight is challenged, the organization can identify whether the issue originates from the quality of the source data, the extraction logic, the analytical thresholds or the interpretation of the query, rather than attributing the failure to an opaque “AI system” as a whole. This granularity supports internal audit processes, facilitates compliance with emerging AI regulations and enables substantive human supervision by HR professionals, legal teams and worker representatives. In this way, the system embeds accountability into the technical workflow in a way that mirrors established organizational governance practices.

With respect to AI governance, the system offers a practical framework for translating abstract principles (such as transparency, explainability and human-in-the-loop control)—into enforceable technical mechanisms. Governance is supported through persistent audit trails, agent-level logging, and the ability to reconstruct decision pathways retrospectively. These features are particularly relevant in light of regulatory frameworks that emphasize documentation, risk management and contestability of

automated decisions [51]. Rather than relying on policy statements or usage guidelines alone, the architecture enforces governance constraints through design (for example, LLM outputs are bounded by retrieved evidence, confidence and uncertainty are made visible and users retain interpretive authority over final decisions). From an organizational perspective, this enables AI systems to be integrated into existing governance regimes (such as ethics committees, works councils or compliance units) without requiring those bodies to trust inscrutable models. More broadly, the work demonstrates how generative AI can be aligned with societal expectations not by reducing its capabilities, but by embedding it within transparent, accountable and governable IS that respect both organizational responsibility and individual rights.

7.3 Limitations and future research

Despite the significant benefits brought forward by the proposed decision-support system, several limitations are worth reporting. While the design is essential for traceability and auditability, it introduces non-trivial storage and query overhead as the volume of organizational data grows. In large-scale or longitudinal HR settings, continuous ingestion and recomputation of analytical facts can lead to rapid graph expansion and increased retrieval latency, even in parallel processing regimes. Future research should, therefore, investigate mechanisms for scalable provenance management, including temporal summarisation, fact versioning and selective materialization strategies that preserve auditability while controlling graph growth. Another limitation relates to the dependence of the system on the quality and normative validity of the HR data upstream. Although the architecture prevents hallucination and exposes the origins of every claim, it does not eliminate bias embedded in historical records, evaluation practices and/or organizational structures. Future work should, therefore, integrate fairness-aware analytics directly into the fact generation and aggregation processes, enabling the system not only to reveal but also to diagnose and potentially mitigate biased patterns before they are operationalized in decision-support outputs. The strictly evidence-constrained generative design of the system is also another limitation. By constraining the LLM to operate exclusively over retrieved, graph-stored facts, the architecture effectively eliminates hallucination and ungrounded inference. However, this constraint also limits the system’s ability to support exploratory, counterfactual, or forward-looking analyzes that extend beyond existing organizational data. Future research could explore controlled extensions of this paradigm, such as explicitly marked hypothetical facts, simulation agents and/or scenario-generation modules, that preserve provenance and accountability while enabling limited forms of counterfactual reasoning. Finally, while the empirical evaluation presented in this work is deliberately rigorous and well-aligned with the paper’s design goals, future research can extend the assessment along two complementary dimensions a) larger-scale empirical evaluation involving a broader pool of evaluators (such as HR practitioners, auditors or policy stakeholders) would enable systematic analysis of inter-rater agreement, usability perceptions and organizational acceptance under real-world conditions and b) applying the proposed architecture to additional HR datasets with different structural properties would further test the robustness and generalizability of the system.

8 Conclusions

This paper demonstrates that transparency, explainability and trust in AI-supported HR decision-making are primarily architectural properties rather than outcomes that can be reliably achieved through post-hoc interpretability techniques. The results show that embedding provenance, accountability and human oversight directly into system design, especially through a provenance-aware KG, a modular multi-agent orchestration layer and a strictly evidence-grounded LLM interface, enables generative AI systems to operate within the evidentiary, reproducibility and governance constraints required in high-stakes organizational contexts. Through scenario-based evaluation on realistic HR analytics tasks, the proposed architecture exhibits deterministic behavior, measurable traceability, resistance to hallucination and low response latency, while deliberately failing conservatively when sufficient evidence is not available. These findings indicate that symbolic structure, agent-level task decomposition and conversational interaction can jointly support reliable and explainable decision support without relying on unrestricted generative inference. Beyond the HR domain, the work contributes to IS and Generative AI research by illustrating that trustworthy AI does not necessarily arise from increasingly complex or opaque models, but from evidence-centric information infrastructures that render reasoning pathways inspectable, contestable and auditable. The main contribution of this paper lies in articulating and operationalizing such an infrastructure, providing a concrete blueprint for governable generative systems. Looking forward, this perspective motivates a research agenda that shifts emphasis away from isolated model performance and toward the systematic design and evaluation of AI systems that align technical capabilities with organizational accountability, fairness and societal expectations.

References

- [1] Li, B., Cheng, Y.: Chatgpt in human resource management: A systematic review of influential factors, processes, and outcomes. *Heliyon* **11**(15), 44048 (2025) <https://doi.org/10.1016/j.heliyon.2025.e44048>
- [2] Li, N., Zhou, H., Xu, M.: From text to insight: Leveraging large language models for performance evaluation in management. *SSRN Electronic Journal* (2024). preprint, arXiv:2408.05328
- [3] Wilson, K., Caliskan, A.: Gender, race, and intersectional bias in resume screening via language model retrieval. In: *Proceedings of the Seventh AAAI/ACM Conference on Artificial Intelligence, Ethics, and Society (AIES 2024)* (2024). preprint arXiv:2407.20371. <https://arxiv.org/abs/2407.20371>
- [4] Dasaklis, T.K., Giannopoulos, P.G., Koutras, D., Malamas, V., Chountalas, P.: Large language models in human resource management: A systematic literature review of applications, open issues and future research directions (2025) <https://doi.org/10.2139/ssrn.5314976> . Available at SSRN
- [5] Andrieux, P., Johnson, R.D., Sarabadani, J., Van Slyke, C.: Ethical considerations

- of generative ai-enabled human resource management. *Organizational Dynamics* **53**(1), 101032 (2024) <https://doi.org/10.1016/j.orgdyn.2024.101032>
- [6] Aguinis, H., Beltran, J.R., Cope, A.: How to use generative ai as a human resource management assistant. *Organizational Dynamics* **53**(1), 101029 (2024) <https://doi.org/10.1016/j.orgdyn.2024.101029>
- [7] Raman, R., et al.: Evaluating human resources management literacy: A performance analysis of chatgpt and bard. *Heliyon* **e27026** (2024)
- [8] Tinguely, P.N., Lee, J., He, V.F.: Designing human resource management systems in the age of ai. *Journal of Organization Design* **12**(1), (2023) <https://doi.org/10.1007/s41469-023-00153-x>
- [9] Wang, Z., Wu, Z., Guan, X., Thaler, M., Koshiyama, A., Lu, S., Beepath, S., Ertekin, E., Perez-Ortiz, M.: JobFair: A framework for benchmarking gender hiring bias in large language models. In: *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 3227–3246. Association for Computational Linguistics, Miami, Florida, USA (2024). <https://doi.org/10.18653/v1/2024.findings-emnlp.184> . <https://aclanthology.org/2024.findings-emnlp.184/>
- [10] Wen, A., Patil, T., Saxena, A., Fu, Y., O’Brien, S., Zhu, K.: FAIRE: assessing racial and gender bias in ai-driven resume evaluations. *arXiv preprint* (2025). [arXiv:2504.01420](https://arxiv.org/abs/2504.01420) [cs.CL]
- [11] Rigotti, C., Fosch-Villaronga, E.: Fairness, ai & recruitment. *Computer Law & Security Review* **53**, 105966 (2024) <https://doi.org/10.1016/j.clsr.2024.105966>
- [12] Frazzetto, P., Haq, M.U.U., Fabris, F., Sperduti, A.: Graph neural networks for candidate-job matching: An inductive learning approach. *Data Science and Engineering* (2025) <https://doi.org/10.1007/s41019-025-00293-y>
- [13] Ye, Y., Zhu, H., Cui, T., Yu, R., Zhang, L., Xiong, H.: University evaluation through graduate employment prediction: An influence based graph autoencoder approach. *IEEE Transactions on Knowledge and Data Engineering* **36**(11), 7255–7267 (2024) <https://doi.org/10.1109/TKDE.2024.3402234>
- [14] Bai, Y., Gong, J., Han, M., Yang, J.: The financial institution text data mining and value analysis model based on big data and natural language processing. *Journal of Organizational and End User Computing* **37**(1) (2025) <https://doi.org/10.4018/JOEUC.374213>
- [15] Yang, Y., Lu, Y., Zheng, Z., Wu, H., Lin, Y., Qian, F., Yan, W.: Mkg-gc: A multi-task learning-based knowledge graph construction framework with personalized application to gastric cancer. *Computational and Structural Biotechnology Journal* **23**, 1339–1347 (2024) <https://doi.org/10.1016/j.csbj.2024.03.021>

- [16] Wang, L., Zhang, J., Zhang, L., Zhang, S., Xu, S., Yu, L., Gao, H.: Mkgfa: Multimodal knowledge graph construction and fact-assisted reasoning for vqa. *International Journal of Computational Intelligence and Applications* **24**(2) (2025) <https://doi.org/10.1142/S1469026824500342>
- [17] Wang, J., Xie, H., Zhang, S., Qin, S.J., Tao, X.D., Wang, F.L., Xu, X.: Multimodal fusion framework based on knowledge graph for personalized recommendation. *Expert Systems with Applications* **268** (2025) <https://doi.org/10.1016/j.eswa.2024.126308>
- [18] Yang, J., Xu, M., Xiao, Y., Du, X.: Amifn: Aspect-guided multi-view interactions and fusion network for multimodal aspect-based sentiment analysis. *Neurocomputing* **573** (2024) <https://doi.org/10.1016/j.neucom.2023.127222>
- [19] Pan, W., Wang, Y., Zhang, Y., Han, B.: Atc-sd net: Radiotelephone communications speaker diarization network. *Aerospace* **11**(7) (2024) <https://doi.org/10.3390/aerospace11070599>
- [20] Bao, Q., Zheng, P., Dai, S.: Hierarchical construction and application of machining domain knowledge graph based on as-fabricated information model. *Advanced Engineering Informatics* **62** (2024) <https://doi.org/10.1016/j.aei.2024.102638>
- [21] Haiyang, X., Ruomei, Y., Yan, W., Guan, L., Li, M.: Knowledge graph for solubility big data: Construction and applications. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* **15**(1) (2025) <https://doi.org/10.1002/widm.1570>
- [22] Ma, L., Tian, X., Huang, L., Yang, F., Shi, X.: A data-driven method combining knowledge graph with deep learning for constructing machining process knowledge base. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture* **239**(9), 1204–1217 (2025) <https://doi.org/10.1177/09544054241272920>
- [23] Xiao, N., Peng, B., Li, X., Wu, J., Lou, J., Si, Y.: Research on the construction and implementation of power grid fault handling knowledge graphs. *Energy Reports* **9**, 182–189 (2023) <https://doi.org/10.1016/j.egyr.2023.02.073>
- [24] Yan, C., Fang, X., Huang, X., Guo, C., Wu, J.: A solution and practice for combining multi-source heterogeneous data to construct enterprise knowledge graph. *Frontiers in Big Data* **6** (2023) <https://doi.org/10.3389/fdata.2023.1278153>
- [25] Yang, M., Namoano, B., Farsi, M., Erkoyuncu, J.A.: Named entity recognition in aviation products domain based on bert. *IEEE Access* **12**, 189710–189721 (2024) <https://doi.org/10.1109/ACCESS.2024.3516390>
- [26] Yang, S., Lu, W., Yang, D., Li, X., Wu, C., Wei, B.: Keyphraseds: Automatic generation of survey by exploiting keyphrase information. *Neurocomputing* **224**,

- 58–70 (2017) <https://doi.org/10.1016/j.neucom.2016.10.052>
- [27] Han, H., Wang, J., Wang, X.: A relation-oriented model with global context information for joint extraction of overlapping relations and entities. *Frontiers in Neurobotics* **16** (2022) <https://doi.org/10.3389/fnbot.2022.914705>
 - [28] Zeng, D., Xu, L., Jiang, C., Zhu, J., Chen, H., Dai, J., Jiang, L.: Sequence tagging with a rethinking structure for joint entity and relation extraction. *International Journal of Machine Learning and Cybernetics* **15**(2), 519–531 (2024) <https://doi.org/10.1007/s13042-023-01923-5>
 - [29] Gao, Y., Gong, G., Li, N.: Chinese relation extraction in military field based on multi-grained lattice transformer and imbalanced data classification. *International Journal of Modeling, Simulation, and Scientific Computing* **15**(5) (2024) <https://doi.org/10.1142/S1793962323500526>
 - [30] Shi, P., Zhang, B., Liu, Y., Fang, C.: Etfre: Entity–type fusing for relation extraction. *Electronics (Switzerland)* **14**(1) (2025) <https://doi.org/10.3390/electronics14010205>
 - [31] Lai, P., Wei, C., Luo, L., Chen, Q., Lu, Z.: Biorex: Improving biomedical relation extraction by leveraging heterogeneous datasets. *Journal of Biomedical Informatics* **146** (2023) <https://doi.org/10.1016/j.jbi.2023.104487>
 - [32] Sung, M., Jeong, M., Choi, Y., Kim, D., Lee, J., Kang, J.: Bern2: an advanced neural biomedical named entity recognition and normalization tool. *Bioinformatics* **38**(20), 4837–4839 (2022) <https://doi.org/10.1093/bioinformatics/btac598>
 - [33] Wang, X., Gan, Z., Xu, Y., Liu, B., Zheng, T.: Extracting domain-specific chinese named entities for aviation safety reports: A case study. *Applied Sciences (Switzerland)* **13**(19) (2023) <https://doi.org/10.3390/app131911003>
 - [34] Yuan, L., Zeng, C., Pan, P.: Research of chinese entity recognition model based on multi-feature semantic enhancement. *Electronics (Switzerland)* **13**(24) (2024) <https://doi.org/10.3390/electronics13244895>
 - [35] Huang, J., Hao, X., Wang, Y., Song, R., Mu, Z., Chu, W., Papadakis, G., Niu, S., Guo, X.: Entity-level cross-modal fusion for multimodal chinese agricultural diseases and pests named entity recognition. *Smart Agricultural Technology* **12** (2025) <https://doi.org/10.1016/j.atech.2025.101188>
 - [36] Fang, Y., Wang, H., Zhao, L., Yu, F., Wang, C.: Dynamic knowledge graph based fake-review detection. *Applied Intelligence* **50**(12), 4281–4295 (2020) <https://doi.org/10.1007/s10489-020-01761-w>
 - [37] Suo, X., Guo, B., Shen, Y., Wang, W., Chen, Y., Zhang, Z.: Embodying the number of an entity’s relations for knowledge representation learning. *International*

- [38] Li, J., Bu, C., Liu, J., Li, P., Wu, X.: Collective entity linking based on evolutionary algorithms: A practical application. *Information Sciences* **719** (2025) <https://doi.org/10.1016/j.ins.2025.122491>
- [39] Dai, Y., Shou, L., Gong, M., Xia, X., Kang, Z., Xu, Z., Jiang, D.X.: Graph fusion network for text classification. *Knowledge-Based Systems* **236** (2022) <https://doi.org/10.1016/j.knsys.2021.107659>
- [40] Li, R., Shu, S., Wang, S., Liu, Y., Li, Y., Peng, M.: Dat-mt accelerated graph fusion dependency parsing model for small samples in professional fields. *Entropy* **25**(10) (2023) <https://doi.org/10.3390/e25101444>
- [41] Shneiderman, B.: *Human-Centered AI*. Oxford University Press, New York, NY (2022). <https://global.oup.com/academic/product/human-centered-ai-9780192845290>
- [42] Deters, H., Droste, J., Obaidi, M., Schneider, K.: A User-Centered Quality Model for Measurable Explainability. *Information and Software Technology* **181**(107682) (2025) <https://doi.org/10.1016/j.infsof.2025.107682>
- [43] Balasubramaniam, N., Kauppinen, M., Rannisto, A., Hiekkänen, K., Kujala, S.: Transparency and Explainability of AI Systems: From Ethical Guidelines to Requirements. *Information and Software Technology* **159**(107197) (2023) <https://doi.org/10.1016/j.infsof.2023.107197>
- [44] Lee, C., Gatt, A., Miltenburg, E., Krahmer, E.: Human evaluation of automatically generated text: Current trends and best practice guidelines. *Computer Speech & Language* **67** (2021) <https://doi.org/10.1016/j.csl.2020.101151>
- [45] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., *et al.*: Training language models to follow instructions with human feedback. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2022)
- [46] Geng, J., Chen, H., Liu, R., Ribeiro, M.H., Willer, R., Neubig, G., Griffiths, T.L.: Accumulating Context Changes the Beliefs of Language Models (2025). <https://arxiv.org/abs/2511.01805>
- [47] Li, X., Jin, J., Zhou, Y., Zhang, Y., Zhang, P., Zhu, Y., Dou, Z.: From matching to generation: A survey on generative information retrieval. *ACM Transactions on Information Systems* **43**(3) (2025) <https://doi.org/10.1145/3722552>
- [48] DeLong, L.N., Mir, R.F., Fleuriot, J.D.: Neurosymbolic ai for reasoning over knowledge graphs: A survey. *IEEE Transactions on Neural Networks and Learning*

- [49] Sado, F., Loo, C.K., Liew, W.S., Kerzel, M., Wermter, S.: Explainable goal-driven agents and robots - a comprehensive review. *ACM Comput. Surv.* **55**(10) (2023) <https://doi.org/10.1145/3564240>
- [50] Novelli, C., Taddeo, M., Floridi, L.: Accountability in artificial intelligence: what it is and how it works. *AI and Society* **39**(4), 1871–1882 (2024) <https://doi.org/10.1007/s00146-023-01635-y>
- [51] Calvano, M., Curci, A., Desolda, G., Esposito, A., Lanzilotti, R., Piccinno, A.: Building symbiotic artificial intelligence: Reviewing the ai act for a human-centred, principle-based framework. *Minds and Machines* **36**(1) (2026) <https://doi.org/10.1007/s11023-025-09753-w>

Declarations

Ethics approval and consent to participate. Not applicable

Consent for publication. All authors consent to this publication.

Availability of data and material. The datasets used and/or analysed during the current study are available from the corresponding author on reasonable request.

Competing interests. The authors have no competing interests to declare relevant to the content of this article.

Funding. This research has been funded by the Hellenic Open University under the project "Multi-agent systems and Generative Artificial Intelligence in Management science: Innovative Applications in Human Resources and Operations management-PELOPAS"

Author contribution. All authors contributed equally to this work.

Appendix A Queries used for performance tests

This appendix provides the complete list of queries used in the experimental evaluation. Queries are grouped by category according to their analytical purpose: *operational*, *strategic*, and *evidence retrieval*.

A.1 Operational Queries

A.1.1 Performance Score by Department

- O1.1 What is the distribution of performance scores by department?
- O1.2 How do performance scores vary across departments?
- O1.3 Which department has the highest average performance score?
- O1.4 Show me performance metrics by department.

A.1.2 Special Projects Count by Department

- O2.1 What is the average special projects count by department?
- O2.2 How do special projects vary across departments?
- O2.3 Which department has the highest average special projects count?
- O2.4 Show me special projects distribution by department.

A.1.3 Engagement by Manager

- O3.1 What is the team-level engagement by manager?
- O3.2 How does engagement vary by manager?
- O3.3 Which manager has the highest team engagement?

A.1.4 Salary by Department

- O4.1 What is the average salary by department?
- O4.2 How does salary distribution vary across departments?
- O4.3 Which department has the highest average salary?

A.1.5 Performance by Recruitment Source

- O5.1 How does performance vary by recruitment source?
- O5.2 Which recruitment sources have employees with the highest performance score?
- O5.3 What is the performance distribution by recruitment source?

A.2 Strategic Queries

A.2.1 Multi-Criteria Employee Search

- S1.1 Identify employees with high performance, low engagement, and many special projects.
- S1.2 Find employees with high performance, low engagement, and low satisfaction.
- S1.3 Find employees with high performance, low engagement, low satisfaction, and many special projects.
- S1.4 Find employees with low engagement, low satisfaction, many special projects, and many absences.

A.2.2 Department–Salary–Performance Analysis

- S2.1 Which departments have high salaries but low performance?
- S2.2 Analyse the relationship between salary, performance, and department.
- S2.3 Identify departments with low salary and high performance.

A.3 Evidence Retrieval Queries

A.3.1 Employee-Specific Fact Retrieval

- E1.1 Retrieve facts related to employee Becker, Scott.
- E1.2 What information do we have about employee Becker, Scott?

A.3.2 Highest Salary Employee

- E2.1 Give me facts about the employee with the highest salary.
- E2.2 Retrieve facts about the employee who has the highest salary.
- E2.3 Show me information about the highest paid employee.

A.3.3 Lowest Performance Employee

- E3.1 Give me facts about the employee with the lowest performance.
- E3.2 Retrieve facts about the employee who has the lowest performance score.
- E3.3 Show me information about the employee with the worst performance.